

Modern Operating Systems
Chapter 2 – Interprocess
Communication

Zhang Yang
Spring 2025

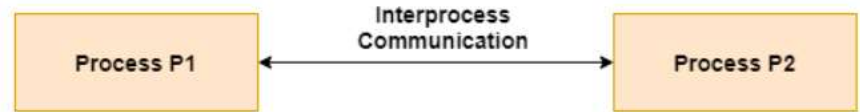


Contents of this lecture

- 2.3 Interprocess Communication
 - Race Conditions
 - Critical Region
 - Mutual Exclusion Using Busy Waiting
 - Sleep and Wakeup
 - Semaphores
 - Monitors
 - Message Passing
- 2.5 Classic IPC Problems
- Interthread Communication
- Summary

- 
- 2.3 Interprocess Communication
 - 2.5 Classic IPC Problems
 - Interthread Communication
 - Summary

Process & IPC



■ Independent Process

- An independent process is not affected by the execution of other processes.

■ Co-operating Process

- A co-operating process can be affected by other executing processes.

■ Inter-Process Communication

- A mechanism that allows processes to communicate with each other and synchronize their actions.
- The communication between these processes can be seen as a method of co-operation between them.



Issues to IPC

- 1. How one process talk to another?
 - Shared memory
 - Message passing
- 2. Make sure that two or more processes do not get in each other's way.
 - Mutual Exclusion
- 3. Proper sequencing
 - Synchronization
- Issues 2 & 3 also apply to threads.

A Race Condition Example (1)

- A function to withdraw money from a bank account:

```
int withdraw (account, amount) {  
    balance = get_balance  
    (account);  
    balance = balance - amount;  
    put_balance (account, balance);  
    return balance;  
}
```

- Now suppose that you and your roommate share a bank account with a balance of ¥1500 (this is not necessarily a good idea...)
 - What happens if you both go to separate ATM machines, and simultaneously withdraw ¥100 from the account?

A Race Condition Example (2)

- We represent the situation by creating a *separate thread* for each ATM user doing a withdrawal.
 - Both threads run on the same bank server system.

Thread 1

```
int withdraw(account, amount) {  
    balance = get_balance(account);  
    balance -= amount;  
    put_balance(account, balance);  
    return balance;  
}
```

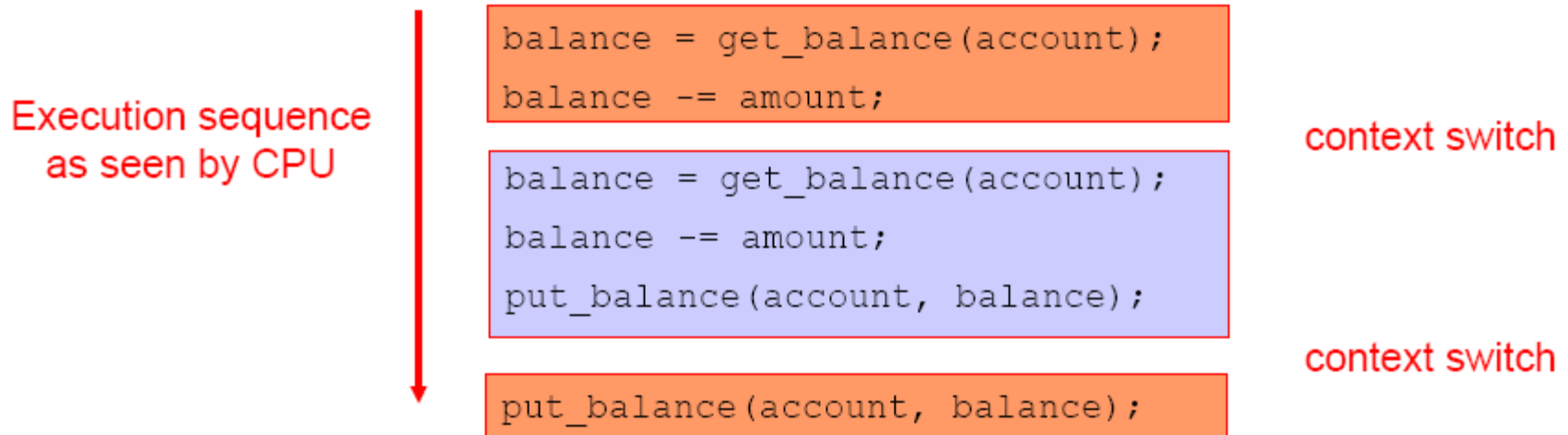
Thread 2

```
int withdraw(account, amount) {  
    balance = get_balance(account);  
    balance -= amount;  
    put_balance(account, balance);  
    return balance;  
}
```

- Assume that threads can interleave executions arbitrarily and at run at different rates. What's the problem with this?
 - What are the possible balance values after each thread runs?

A Race Condition Example (3)

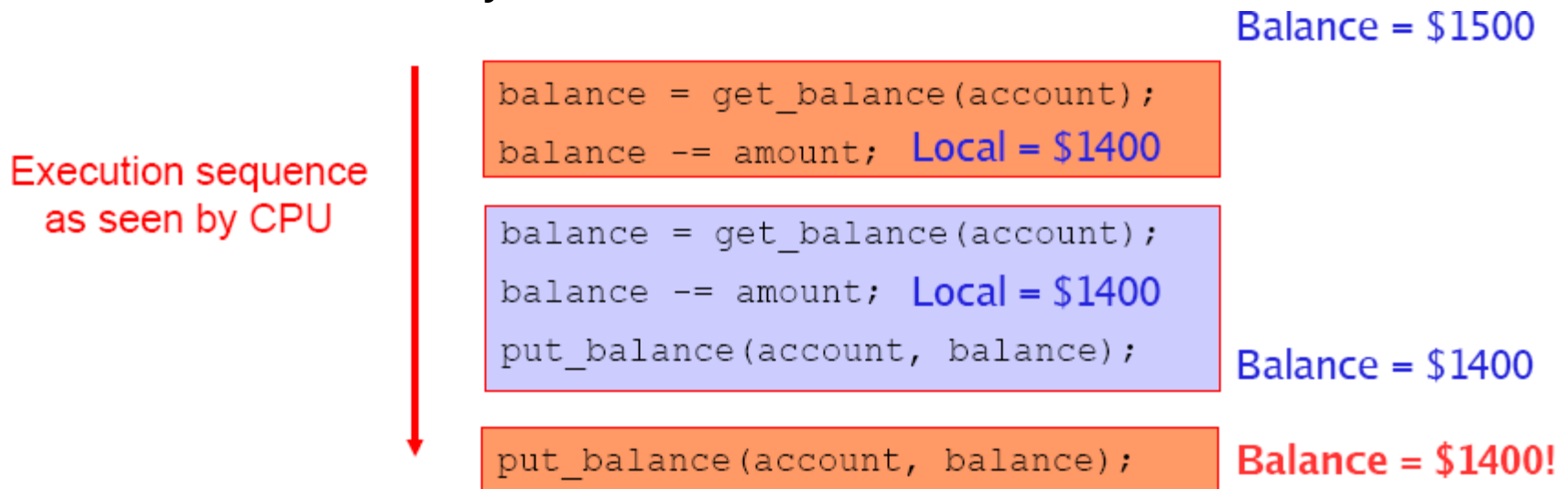
- The execution of the two threads can be *interleaved*.
 - Assume preemptive scheduling.
 - Each thread can context switch after *each* instruction.
 - We need to worry about the worst-case scenario!



- What's the account balance after this sequence?
 - And who's happier, the bank or you???

A Race Condition Example (4)

- The execution of the two threads can be *interleaved. (ctd.)*
 - Assume preemptive scheduling
 - Each thread can context switch after *each* instruction
 - We need to worry about the worst-case scenario!

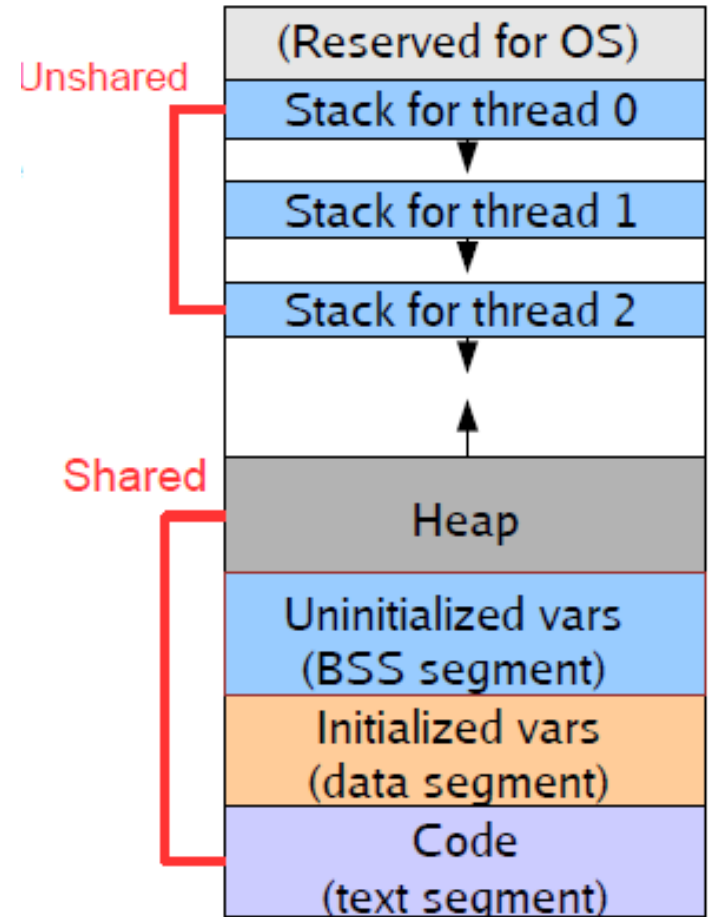


- What's the account balance after this sequence?
 - And who's happier, the bank or you???

A Race Condition Example (5)

■ Which resources are shared?

- Local variables in a function are not shared.
 - They exist on the stack, and each thread has its own stack.
- Global variables are shared.
 - Stored in static data portion of the address space.
 - Accessible by any thread.
- Dynamically allocated data is shared.
 - Stored in the heap, accessible by any thread.



A Race Condition Example (6)

- The problem is that two concurrent threads access a shared resource without any synchronization.
 - The result of the concurrent access is **non-deterministic**.
 - Result depends on:
 - *Timing*
 - *When context switches occurred*
 - *Which thread ran at context switch*
 - *What the threads were doing*
- We need mechanisms for controlling access to shared resources in the face of concurrency.
 - This allows us to reason about the operation of programs.
 - Essentially, we want to re-introduce **determinism** into the thread's execution.
- Synchronization is necessary for any shared data structure.
 - Buffers, queues, lists, hash tables, ...

Race Condition

■ Race Condition

- The situation where several processes access and manipulate **shared data concurrently**. The final value of the shared data depends upon which process finishes **last**.

■ To prevent race conditions

- Shared resources: mutual exclusion
- Synchronization
 - The coordination of simultaneous processes or threads to complete a task in order to get correct runtime order and avoid unexpected race conditions.

Critical Region (1)

■ Critical Section (Region)

- A code segment that accesses shared variables (or other shared resources) and that has to be executed as an atomic action.

Process {

while (true) {

entry-Section // enter critical section

Critical Section // contains accesses to shared
variables or other resources

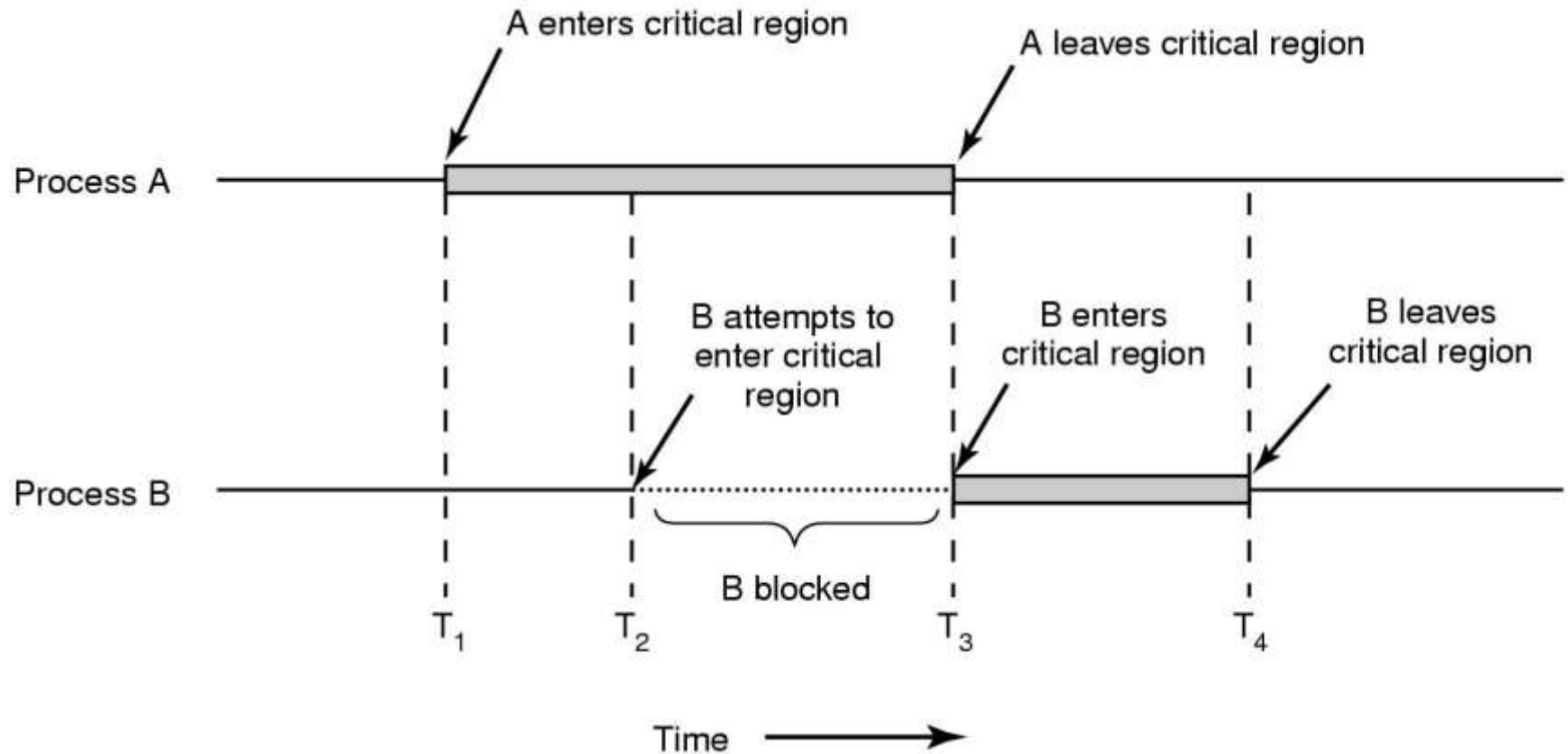
exit-Section // leave critical section

noncritical section //Do other work

}

}

Critical Region (2)



Mutual exclusion using critical regions

Critical Region Requirements(1)

■ Mutual Exclusion Safety

- No other process must execute within the critical section while a process is in it.

■ Progress Liveness

- If no process is executing in its critical section and there are processes that wish to enter their critical sections, then only the processes that are executing in their entry- or exit-sections can participate in the decision about which process will enter its critical section next, and this decision cannot be postponed indefinitely.
- I.e. No process running outside its critical region may block other processes.

Critical Region Requirements(2)

■ Bounded Waiting Fairness

- There exists a limit as to how many other processes can get into their critical sections after a process requests entry into their critical section and before that request is granted.
- I.e. **No process should have to wait forever to enter its critical section.**

■ Speed and Number of CPU Portability

- No assumption may be made about speeds or number of CPUs.



Mutual Exclusion With Busy Waiting

■ Possible Solutions

□ Software Solutions

- Lock Variables
- Strict Alternation
- Peterson's Solution

□ Hardware Solutions

- Disabling Interrupts
- TSL

Lock Variables (1)

shared int lock = 0;

while (lock) { / spin spin spin spin */ }*

lock = 1;

EnterCriticalSection;

access shared variable;

LeaveCriticalSection;

lock = 0;

→ The caller waits for the lock to be released.

■ What's the problem?

Lock Variables (2)

■ Example

```
int withdraw (account, amount) {  
    balance = get_balance (account);  
    balance -= amount;  
    put_balance (account, balance);  
    return balance;  
}
```

critical section

Question: Why is the “return” statement outside of the critical section?

Lock Variables (3)

■ Example (ctd.)

- What happens when the blue thread tries to acquire the lock?



```
While (lock) { }  
lock = 1;  
balance = get_balance(account);  
balance -= amount;
```

lock = 0

Thread 1 runs

```
While (lock) { }
```

Thread 2 waits on lock

```
put_balance (balance,account);  
lock = 0;
```

Thread 1 completes

```
lock = 1;  
balance = get_balance(account);  
balance -= amount;  
put_balance (balance,account);  
lock = 0;
```

Thread 2 resumes

Lock Variables (4)

■ Example (ctd.)

- What happens when the blue thread tries to acquire the lock? (ctd.)

While(lock){ }

lock = 0

Thread 1 runs

while(lock){ }

Thread 2 runs

lock = 1;
balance = get_balance(account);
balance -= amount;

Thread 1 runs

lock = 1;
balance = get_balance(account);
balance -= amount;

Thread 2 runs

...

Violates
mutual
exclusion

Lock Variables (5)

■ Example (ctd.)

- Problem is that the internals of the lock **acquire/release** have critical sections too!

```
while (lock) { /* spin spin spin spin */ }
```

```
lock = 1;
```

```
EnterCriticalSection;
```

```
access shared variable;
```

```
LeaveCriticalSection;
```

```
lock = 0;
```

What can happen if there is a context switch here?

Two processes will be in their critical region at the same time!

These sequence needs to be atomic.

Atomic means that the code cannot be interrupted during execution

Strict Alternation (1)

```
Process /* For two processes */  
{  
  while (true)  
  {  
    while ( turn != my_process_id) { };  
    access shared variables;  
    turn = other_process_id;  
    do other work;  
  }  
}
```

Strict Alternation (2)

Process 0

```
while (TRUE) {  
    while (turn != 0)      /* loop */ ;  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

(a)

Process 1

```
while (TRUE) {  
    while (turn != 1)      /* loop */ ;  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

(b)

The initial value of turn is 0.

Satisfies mutual exclusion? bounded waiting?
progress?

Why?

Strict Alternation (3)

- Question: What happens when Process 1 is slow in executing?
- Solution:
 - Process 0 eventually is blocked waiting on Process 1 to complete it's noncritical section.
 - This violates our condition of no process running outside its critical region may block another process.

Peterson's Solution (1)

- A classic software-based solution to the critical-section problem.
- Does not require strict alternation.
- Restricted to two processes that alternate execution between their CSs and remainder sections.

process i

```
enter_region ( i );
```

```
Critical region
```

```
leave_region ( i );
```

```
Noncritical region
```

Peterson's Solution (2)

```
#define FALSE 0
#define TRUE 1
#define N      2                /* number of processes */

int turn;                       /* whose turn is it? */
int interested[N];              /* all values initially 0 (FALSE) */

void enter_region(int process);  /* process is 0 or 1 */
{
    int other;                  /* number of the other process */

    other = 1 - process;        /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;             /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)   /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

It works!!! Meets all three requirements; solves the critical-section problem for **two** processes. Why?

Peterson's Solution (3)

- Mutual exclusion: Enter critical section if and only if
 - Other process does not want to enter.
 - Other process wants to enter, but your turn.
- Progress: Both processes cannot wait forever at while() loop
 - An uninterested process will never stop the other interested process from entering in the critical section. If the other process is also interested then the process will wait.
- Bounded waiting
 - Each process waits at most one critical section.

Synchronization Hardware

- Many systems provide hardware support for critical section code.
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption.
 - Generally too inefficient on multiprocessor systems.
 - Operating systems using this not broadly scalable.
- Modern machines provide special atomic hardware instructions.
 - Atomic = non-interruptable
 - Either test memory word and set value.
 - Or swap contents of two memory words.

Disabling Interrupts (1)

process A

```
...  
disable interrupts  
CS  
enable interrupts
```

process B

```
...  
disable interrupts  
CS  
enable interrupts
```

■ How does it work?

- Disable all interrupts just after entering a critical section and re-enable them just before leaving it.

■ Why does it work?

- With interrupts disabled, no clock interrupts can occur. (The CPU is only switched from one process to another as a result of clock or other interrupts, and with interrupts disabled, no switching can occur.)

Disabling Interrupts (2)

■ Problems

- ☐ What if the process forgets to enable the interrupts?
- ☐ Multiprocessor?
 - Disabling interrupts only affects one CPU.
- Not a good solution for user programs (too powerful and not flexible).
- Only used inside OS.

Test-and-Set Lock (TSL) (1)

- The hardware must support a special instruction, TSL, which does 2 things in a single **atomic** action:

TSL RX, LOCK

- (a) Read the contents of the memory word LOCK into register RX.
- (b) Store a nonzero value at the memory address LOCK.

Test-and-Set Lock (TSL) (2)

enter_region:

TSL REGISTER, LOCK	copy lock to register and set lock to 1
CMP REGISTER, #0	was lock zero?
JNE enter_region	if it was non zero, lock was set, so loop
RET	return to caller; critical region entered

leave_region:

MOVE LOCK, #0	store a 0 in lock
RET	return to caller

Figure 2-25. Entering and leaving a critical region using the TSL instruction.

Test-and-Set Lock (TSL) (3)

■ XCHG Instruction

- Exchange the contents of two locations atomically.
- All Intel x86 CPUs use XCHG instruction for low-level synchronization.

```
enter_region:
    MOVE REGISTER,#1          | put a 1 in the register
    XCHG REGISTER,LOCK        | swap the contents of the register and lock variable
    CMP REGISTER,#0           | was lock zero?
    JNE enter_region          | if it was non zero, lock was set, so loop
    RET                       | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0              | store a 0 in lock
    RET                       | return to caller
```

Figure 2-26. Entering and leaving a critical region using the XCHG instruction.

Problem with CS Solutions (1)

■ The CS Problem: Competition

- Processes compete for a resource that may not be accessed by more than one process at any given time.
- Each process could exist without the other-their interaction is needed only to solve simultaneous access to the resource.

■ Problems of Peterson's & TSL & XCHG Solution

- Busy Waiting
 - Wasting CPU

Problem with CS Solutions (2)

■ Problems of Peterson's & TSL & XCHG Solution (ctd.)

□ Priority Inversion Problem

- A high priority waits for a low priority to leave the critical section.
 - Consider a computer with two processes, H, with high priority and L, with low priority.
 - The scheduling rules are such that H is run whenever it is in ready state.
 - At a certain moment, with L in its critical region, H becomes ready to run (e.g., an I/O operation completes).
 - H now begins busy waiting, but since L is never scheduled while H is running, L never gets the chance to leave its critical region,
 - so H loops forever.

Sleep and Wakeup (1)

- Prevent Busy Waiting with Blocking System Calls: Sleep and Wakeup Primitives
 - Block instead of busy waiting when it is not allowed to enter the Critical Region. (Sleep)
 - Wakeup when it is OK to retry entering the critical region.
- Primitive
 - A segment of code performing a specific function.
 - Implemented by atomic operations.
 - Indivisible
 - Non-interruptable

Sleep and Wakeup (2)

■ Sleep

- A system call that causes the caller to block, that is, be suspended until another process wakes.

■ Wakeup

- Called by a process when it leaves its critical section if any other processes have blocked.

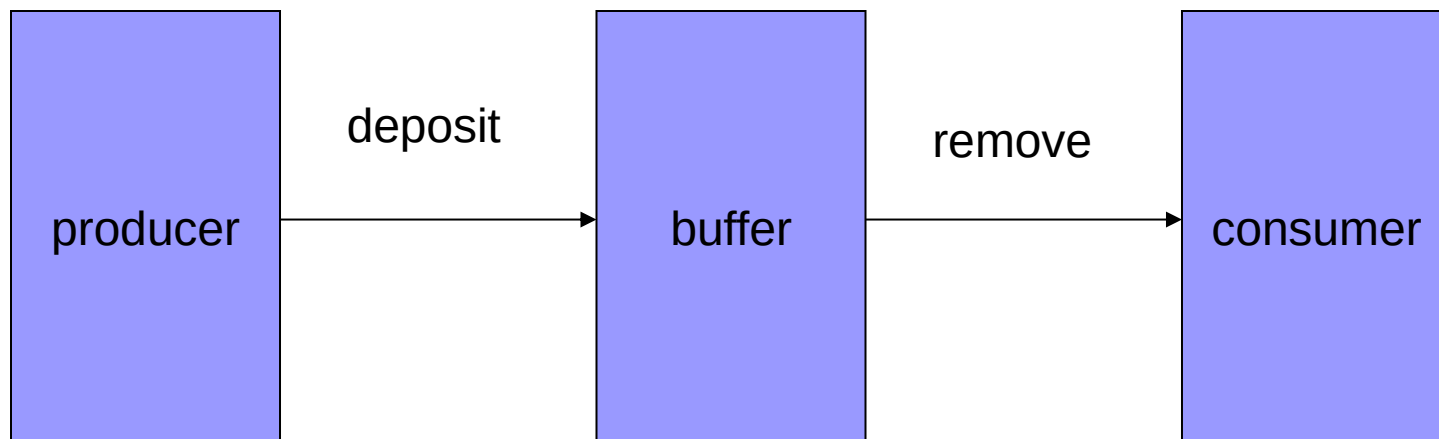
Producer-Consumer Problem (1)

■ Cooperation

- Two or more processes cooperate in solving a common goal.

■ Problem Definition (bounded buffer problem)

- Producer puts things into a shared buffer (wait if full)
- Consumer takes them out (wait if empty)
- Use a fixed-size buffer between them to avoid lockstep.
 - Need to synchronize to this buffer.



Producer-Consumer Problem (2)

■ Correctness Constraints

- Producer must wait for consumer to empty buffers, if all full (scheduling constraint)
- Consumer must wait for producer to fill buffers, if none full (scheduling constraint)
- Only one process can manipulate buffer queue at a time (mutual exclusion)

■ Solution

- The producer goes to sleep if the buffer is full. The next time the consumer removes an item from the buffer, it wakes up the producer who starts to fill the buffer again.
- The consumer goes to sleep if it finds the buffer to be empty. The next time the producer puts data into the buffer, it wakes up the sleeping consumer.

Producer-Consumer Problem (3)

```
#define N 100                                     /* number of slots in the buffer */
int count = 0;                                   /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {                               /* repeat forever */
        item = produce_item();                  /* generate next item */
        if (count == N) sleep();                 /* if buffer is full, go to sleep */
        insert_item(item);                       /* put item in buffer */
        count = count + 1;                       /* increment count of items in buffer */
        if (count == 1) wakeup(consumer);        /* was buffer empty? */
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {                               /* repeat forever */
        if (count == 0) sleep();                 /* if buffer is empty, got to sleep */
        item = remove_item();                    /* take item out of buffer */
        count = count - 1;                       /* decrement count of items in buffer */
        if (count == N - 1) wakeup(producer);    /* was buffer full? */
        consume_item(item);                      /* print item */
    }
}
```



What happens if consumer
is interrupted after reading
count as 0?

Figure 2-27 Producer-consumer problem with fatal race condition

Producer-Consumer Problem (4)

- Race condition can lead to deadlock.
 1. The consumer has just read the variable *count*, noticed it's zero and is just about to move inside the if-block.
 2. Just before calling sleep, the consumer is interrupted and the producer is resumed.
 3. The producer creates an item, puts it into the buffer, and increases *count*.
 4. Because the buffer was empty prior to the last addition, the producer tries to wake up the consumer.
 5. Unfortunately the consumer wasn't yet sleeping, and the wakeup call is lost. The next time the consumer runs, it goes to sleep and will never be awakened.
 6. The producer will loop until the buffer is full, after which it will also go to sleep.

Producer-Consumer Problem (5)

■ Solution

- Add a wakeup waiting bit.

- When a wake up is sent to a process that is still awake, this bit is set to 1.
- When the process tries to go to sleep, if the wakeup bit is on, it will be turn off, but the process is still awake.

What is a Semaphore? (1)

- Semaphores introduced to keep track of “wakeups”.
- Semaphores are system-implemented data structures that are shared between processes.
- Designed by E.W.Dijkstra in 1965 as part of the THE operating system.
- Higher-level synchronization mechanism.
- Higher than disabling interrupts.
 - **Fine** for short sequences of code in **kernel**.
 - **Not fine** for application-level use.
- Fundamental Principle
 - Two or more processes want to cooperate by means of simple signals.

What is a Semaphore? (2)

- A semaphore, S, is a structure consisting of two parts:
 - (a) an integer counter, count
 - (b) a FIFO queue of pids of blocked processes, Q

```
struct sem_struct {  
    int count;  
    queue Q;  
} semaphore;
```

```
semaphore S;
```

Operations of Semaphores (1)

■ Two Operations

- P(sem S) or wait(sem S) or down (sem S)
 - From Dutch “*proeberen*”, meaning “test to decrement”.
 - Is used to acquire a resource and decrements count.
- V(sem S) or signal(sem S) or up(sem S)
 - From Dutch “*verhogen*”, meaning “increment”
 - Is used to release a resource and increments count.
- Any semaphore operation is **indivisible**, must be executed **atomically**.

Operations of Semaphores (2)

■ P/wait/down(sem)

- If sem is “available” (>0), decrement sem; let process continue.
- Otherwise, place process on associated queue; run some other process.

■ V/signal/up(sem)

- If processes are waiting on the associated queue, unblock one (place it on the ready queue).
- If no processes are on the queue, sem is incremented.
- The signal is “remembered” for next time P(sem) is called.

Operations of Semaphores (3)

- How do we ensure that the semaphore primitives is atomic?
 - Approach 1
 - Make them system calls, and ensure only one P() or V() operation can be executed by any process at a time.
 - This effectively puts a **lock** around the P() and V() operations themselves!
 - Easy to do by disabling interrupts in the P() and V() calls.
 - Approach 2
 - Use hardware support.
 - Say your CPU had atomic P and V instructions.

Semaphore Operation Exercise (1)

- The following figure shows the sequence of semaphore operations at the beginning and at the end of the tasks A, B, C.

Task A	Task B	Task C
P(SA)	P(SB)	P(SC)
P(SA)	.	P(SC)
P(SA)	.	P(SC)
.	.	.
.	.	.
.	V(SC)	V(SB)
V(SB)	V(SA)	V(SB)
END	END	END

Semaphore Operation Exercise (2)

- Determine for the 4 cases a), b), c), d), given in the following table, whether and in which sequence the tasks are executed, using the initializations of the semaphore variables given in the table.

variable	a)	b)	c)	d)
SA	2	3	2	0
SB	0	0	1	0
SC	2	2	1	3

Semaphore Operation Exercise (3)

■ Solution

	SA	SB	SC	Executable Task	Remarks
a) initial values	2	0	2	none	None of the tasks is executable. All tasks are blocked! (deadlock)
b) initial values	3	0	2	A	Sequence: ABCBB ABCBB ...
	0	1	2	B	
	1	0	3	C	
	1	2	0	B	
	2	1	1	B	
	3	0	2		

Uses of Semaphores

- Critical Section Guard (initial value: 1)
 - Semaphores that control access to a single resource, taking the value of 0 (resource is in use) or 1 (resource is available), are often called *binary semaphores*.
- Precedence Enforcer (initial value: 0)
 - Semaphores ensuring an ordering of execution among concurrent processes
- Resource Counter (initial value: N)
 - Semaphores controlling access to N (multiple) resources, thus assuming a range of non-negative values, are frequently called *counting semaphores*.

Two Types of Semaphores

- Binary Semaphore (0,1) (aka mutex semaphore)
 - **sem is initialized to 1.**
 - Guarantees mutually exclusive access to resource (e.g., critical section).
 - Only one process/thread allowed entry at a time.
 - Can be simpler to implement.
- Counting Semaphore (0..N)
 - **sem is initialized to N.**
 - N = number of units available
 - Represents resources with many (identical) units available.
 - Allows processes/threads to enter as long as more units are available.

Producer and Consumer Problem Using Semaphores (1)

必须置一次且
只能置一次初值
初值不能为负数

```
semaphore mutex = 1;  
semaphore empty = N;  
semaphore full = 0;
```

```
/* controls access to critical region */  
/* counts empty buffer slots */  
/* counts full buffer slots */
```

```
void producer(void)  
{
```

```
    int item;
```

```
    while (TRUE) {  
        item = produce_item();  
        down(&empty);  
        down(&mutex);  
        insert_item(item);  
        up(&mutex);  
        up(&full);  
    }
```

```
/* TRUE is the constant 1 */  
/* generate something to put in buffer */  
/* decrement empty count */  
/* enter critical region */  
/* put new item in buffer */  
/* leave critical region */  
/* increment count of full slots */
```

```
void consumer(void)  
{
```

```
    int item;
```

```
    while (TRUE) {  
        down(&full);  
        down(&mutex);  
        item = remove_item();  
        up(&mutex);  
        up(&empty);  
        consume_item(item);  
    }
```

```
/* infinite loop */  
/* decrement full count */  
/* enter critical region */  
/* take item from buffer */  
/* leave critical region */  
/* increment count of empty slots */  
/* do something with the item */
```

```
    }  
}
```

Producer and Consumer Problem Using Semaphores (2)

■ Group Discussion

- 会不会出现生产者和消费者同时睡觉而造成死锁？
- 为什么需要三个信号量？一个信号量用来互斥，一个信号量用来记录缓冲区里的商品数量不就可以了么？缓冲区里的空格数量不是可以由缓冲区大小和缓冲区里的商品数量计算出来么？为什么需要一个full和empty来记录满的和空的呢？

Producer and Consumer Problem Using Semaphores (3)

- Why asymmetry?

- ☐ Producer does: `down(&empty), up(&full)`
- ☐ Consumer does: `down(&full), up(&empty)`

- Is order of P's important?

Yes! Can cause deadlock.

- Is order of V's important?

No, except that it might affect scheduling efficiency.

- What if we have 2 producers or 2 consumers?

- ☐ Do we need to change anything?

信号量及P、V操作讨论

- 信号量S的物理含义
 - $S > 0$ 表示有S个资源可用
 - $S = 0$ 表示无资源可用
 - $S < 0$ 则 $|S|$ 表示S等待队列中的进程个数
 - $P(S)$: 表示申请一个资源
 - $V(S)$: 表示释放一个资源。
 - 信号量的初值应该大于等于0
- P、V操作必须成对出现，有一个P操作就一定有一个V操作。当为互斥操作时，它们同处于同一进程，当为同步操作时，则不在同一进程中出现。
- 如果 $P(S1)$ 和 $P(S2)$ 两个操作在一起，那么P操作的顺序至关重要。一个同步P操作与一个互斥P操作在一起时，同步P操作在互斥P操作前，而两个V操作无关紧要。

Semaphore as Synchronization Tool (1)

■ Process Synchronization: Order process execution

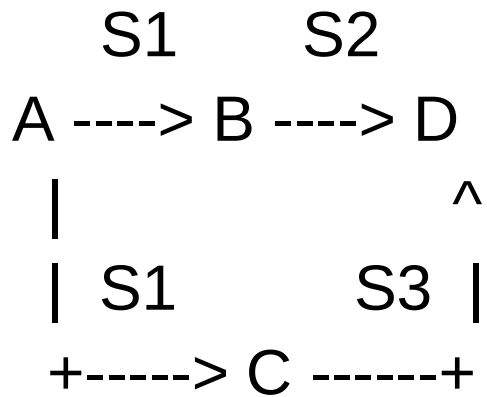
- Want to execute B in P_j only after A executed in P_i
- Use semaphore S initialized to 0
- Code:

Semaphore $S=0$; // initial value of semaphore = 0

P_i		P_j
A		$P(S)$
$V(S)$		B

Semaphore as Synchronization Tool (2)

- Example: Suppose we have 4 processes: A, B, C, and D. A must finish executing before B and C start. B and C must finish executing before D starts.



Then, the processes may be synchronized using semaphores:

```
semaphore S1, S2, S3 = 0,0,0;
```

Semaphore as Synchronization Tool (3)

■ Example Solution

Process A:

- do work of A

V(S1); /* Let B or C start */

V(S1)

Process B:

P(S1); /* Block until A is finished */

- do work of B

V(S2);

Semaphore as Synchronization Tool (4)

■ Example Solution (ctd.)

Process C:

P(S1);

- do work of C

V(S3);

Process D:

P(S2);

P(S3);

- do work of D

Semaphore Synchronization Exercise (1)

■ 司机-售票员问题

□ 设公共汽车上，司机和售票员的活动分别是：

司机：

启动车辆

正常行车

到站停车

售票员：

关车门

售票

开车门

□ 在汽车不断到站，停车，行驶过程中，这两个活动的同步关系？

Semaphore Synchronization Exercise (2)

■ 司机-售票员问题（续）

□ 分析

- 第一步:确定进程间的关系。司机到站停车后,售票员方可工作。同样,售票员关车门后,司机才能工作。所以司机与售票员之间是一种同步关系。
- 第二步:确定信号量及其值。由于司机与售票员之间要互通消息,司机进程设置一个信号量是s2,用于判断司机能否进行工作,初值为0。售票员进程设置一个信号量s1,用于判断是否停车,售票员是否能够开车门,初值为0。

Semaphore Synchronization Exercise (3)

■ 司机-售票员问题 （续）

```
struct semaphore  
    s1,s2=0,0;
```

```
void driver(void)  
{  
    while(TRUE){  
        P(s2);  
        启动车辆;  
        正常行车;  
        到站停车;  
        V(s1);  
    }
```

```
void conductor(void)  
{  
    while(TRUE){  
        关车门;  
        V(s2);  
        售票;  
        P(s1);  
        开车门; }  
}
```


Problems with Semaphores (1)

- Semaphores can be used to solve any of the traditional synchronization problems, but:
 - Semaphores are essentially shared global variables. They can be accessed from anywhere (bad software engineering).
 - There is no connection between the semaphore and the data being controlled by it. No control over their use, no guarantee of proper usage.
 - Thus, semaphores are prone to bugs. Another (better?) approach
 - Use programming language support.

Problems with Semaphores (2)

■ Example

What's the problem?
Deadlock!

Assume that
empty = 0, then
the producer
would block,
mutex = 0. The
next time, the
consumer will
block too.

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

exchange ↗

/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */

/* TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of full slots */

/* infinite loop */
/* decrement full count */
/* enter critical region */
/* take item from buffer */
/* leave critical region */
/* increment count of empty slots */
/* do something with the item */

What is a Monitor?

- A collection of procedures, variables, and data structures that are all grouped together in a special kind of module or package.
- Mutual Exclusion
 - Allows controlled acquisition and release of critical resources.
 - Single anonymous lock automatically acquired and released at entry and exit.
- Data Encapsulation
 - Monitor procedures are “entry points” for accessing data.
- Monitors are a programming language construct.
- Java, C#, etc., have built-in support for monitors.

An Abstract Monitor (1)

name: monitor

...local declarations

...initialize local data

proc1 (...parameters)

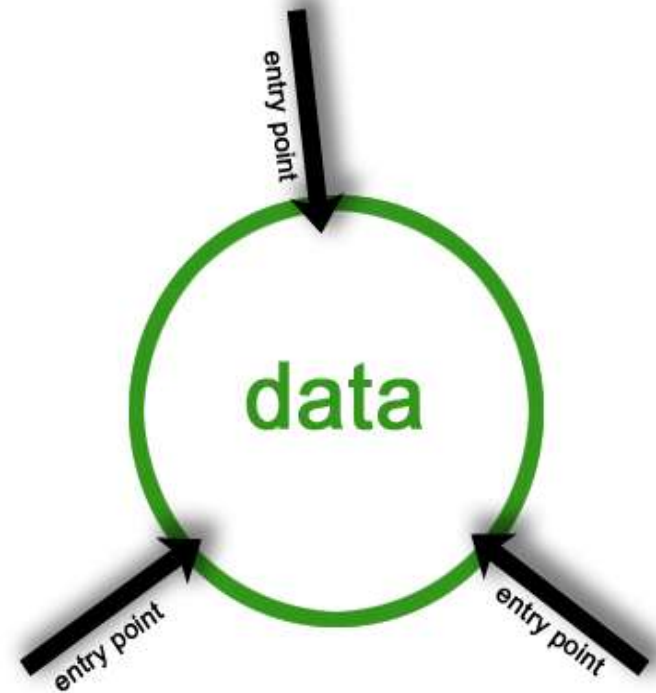
...statement list

proc2 (...parameters)

...statement list

proc3 (...parameters)

...statement list



An Abstract Monitor (2)

■ Monitor Example by Pidgin Pascal

```
monitor example
  integer i;
  condition c;

  procedure producer( );
    .
    .
    .
  end;

  procedure consumer( );
    .
    .
    .
  end;
end monitor;
```

Monitors (1)

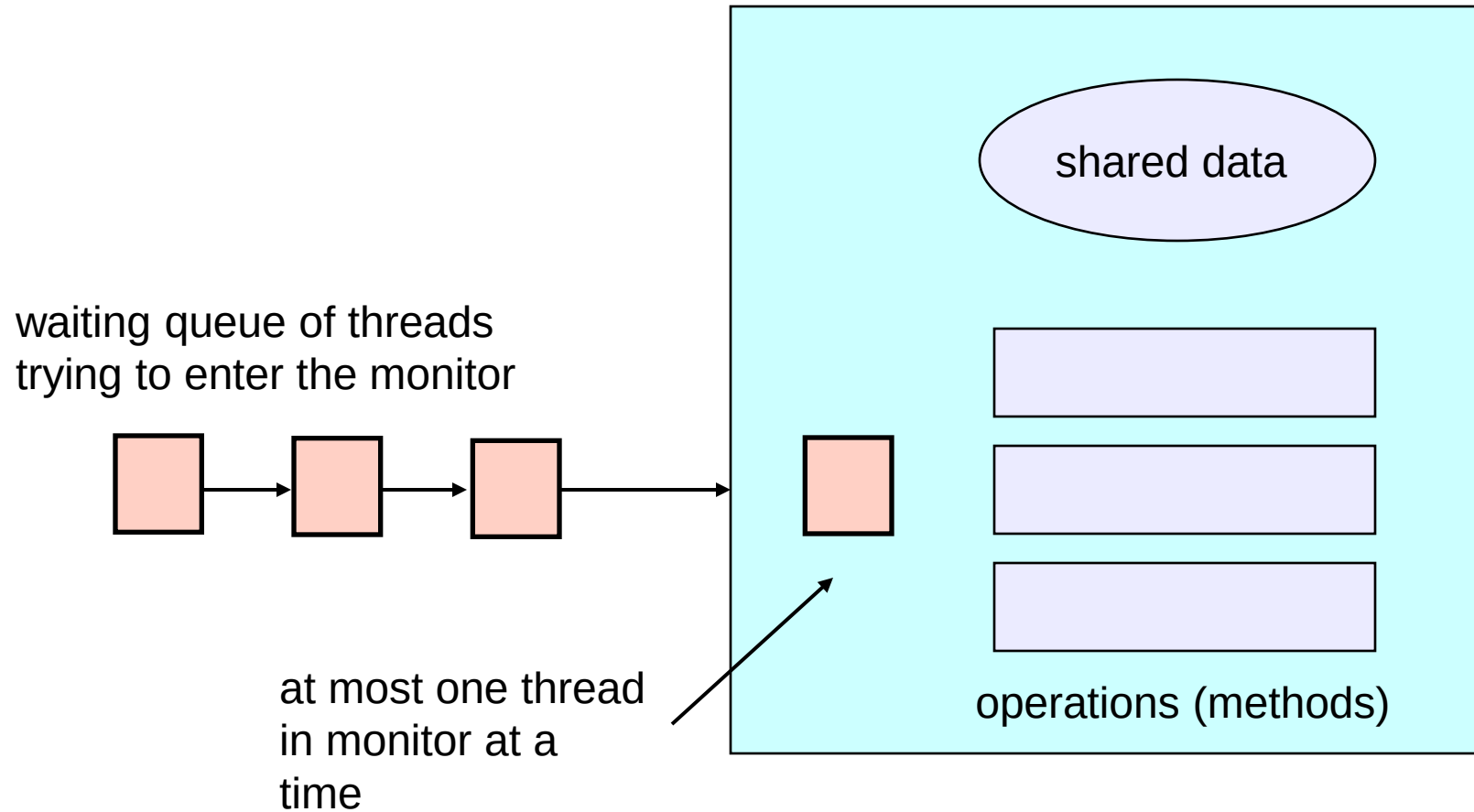
- A monitor encapsulates
 - Shared data structures.
 - Procedures that operate on the shared data.
 - Synchronization between concurrent processes/threads that invoke those procedures.
- General Approach
 - Collect related shared data into an object/module.
 - Define methods for accessing the shared data.

Monitors (2)

■ Rules to Follow with Monitors

- A process/thread enters the monitor by invoking one of its procedures.
- **Only one** process/thread can be **active** within the monitor at a time.(mutual exclusion).
 - Any other process that has invoked the monitor is suspended, waiting for the monitor to become available.
- No process can directly access a monitor's local variables. (data encapsulation)
- A monitor may only access its local variables.

Monitors (3)



Monitors (4)

■ Mutual Exclusion

- The monitor ensures mutual exclusion - no need to program this constraint explicitly. (compiler)
- The monitor **locks** (protects) shared data on process entry.

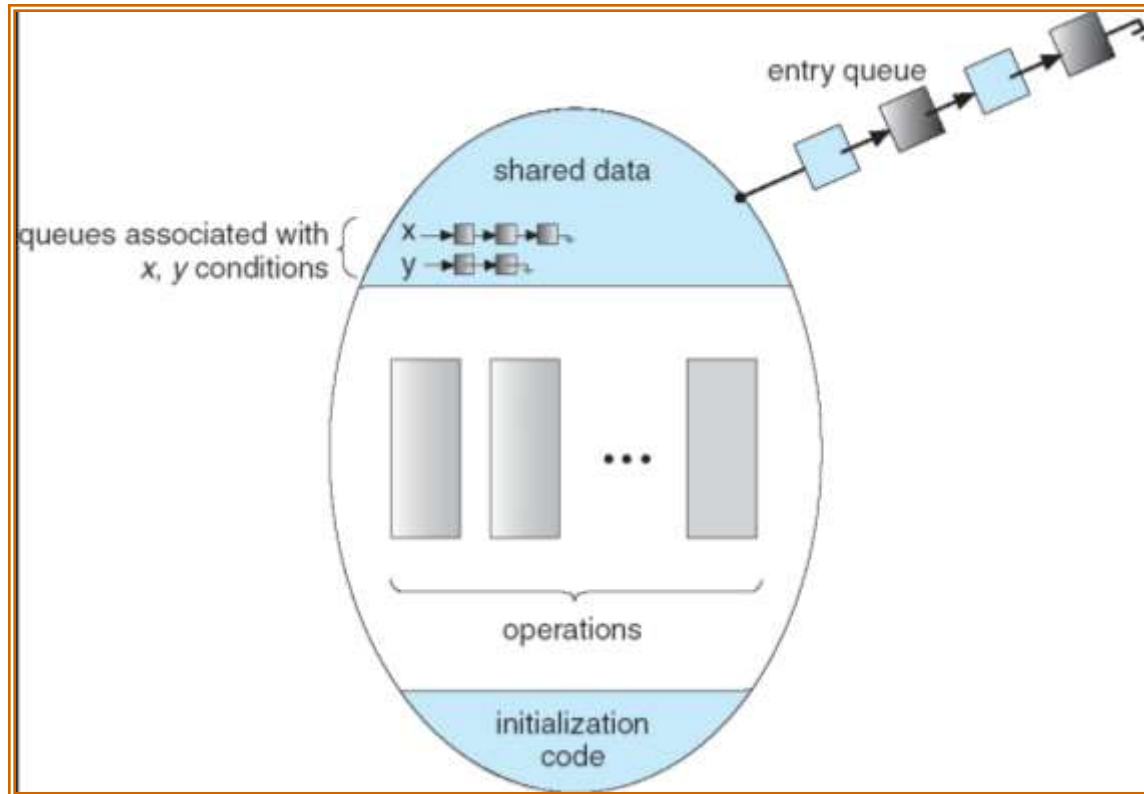
■ Process Synchronization

- Done by the programmer by using **condition variables**.
- Conditions need to be satisfied before entering monitor.

Condition Variables (1)

- A place to wait. Sometimes called a rendezvous point. (会合点)
- Local to the monitor.
 - Accessible only within the monitor.
- A queue of processes/threads waiting for something *inside* a critical section.
- Contrast to semaphores: Can't wait inside critical section.

Condition Variables (2)



Monitor with Condition Variables

Operations on Condition Variables

■ wait(c)

- 1. **Release** monitor lock, so somebody else can get in.
- 2. Suspend the process/thread on condition variable c's wait queue.
- 3. Wait for somebody else to signal condition variable c.
- **Note that a process/thread must be holding lock when it calls wait.**

■ signal(c)

- Wake up at most one waiting process/thread. If several such processes/threads exist, choose any one.
- If no waiting processes/threads, signal is lost. This is different than semaphores: no history!

■ broadcast(c)

- Wake up all waiting processes/threads.

Producer-Consumer Problem with Monitors

```
monitor ProducerConsumer
  condition full, empty;
  integer count;
  procedure insert(item: integer);
  begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty)
  end;
  function remove: integer;
  begin
    if count = 0 then wait(empty);
    remove = remove_item;
    count := count - 1;
    if count = N - 1 then signal(full)
  end;
  count := 0;
end monitor;
```

```
procedure producer;
begin
  while true do
    begin
      item = produce_item;
      ProducerConsumer.insert(item)
    end
  end;
procedure consumer;
begin
  while true do
    begin
      item = ProducerConsumer.remove;
      consume_item(item)
    end
  end;
end;
```

■ Figure 2-34 Outline of producer-consumer problem with monitors

Recall: Sleep and Wakeup

```
#define N 100                                     /* number of slots in the buffer */
int count = 0;                                   /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {                               /* repeat forever */
        item = produce_item();                  /* generate next item */
        if (count == N) sleep();                /* if buffer is full, go to sleep */
        insert_item(item);                      /* put item in buffer */
        count = count + 1;                      /* increment count of items in buffer */
        if (count == 1) wakeup(consumer);      /* was buffer empty? */
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {                               /* repeat forever */
        if (count == 0) sleep();                /* if buffer is empty, got to sleep */
        item = remove_item();                   /* take item out of buffer */
        count = count - 1;                      /* decrement count of items in buffer */
        if (count == N - 1) wakeup(producer);  /* was buffer full? */
        consume_item(item);                     /* print item */
    }
}
```

Figure 2-27 Producer-consumer problem with fatal race condition



Group Discussion

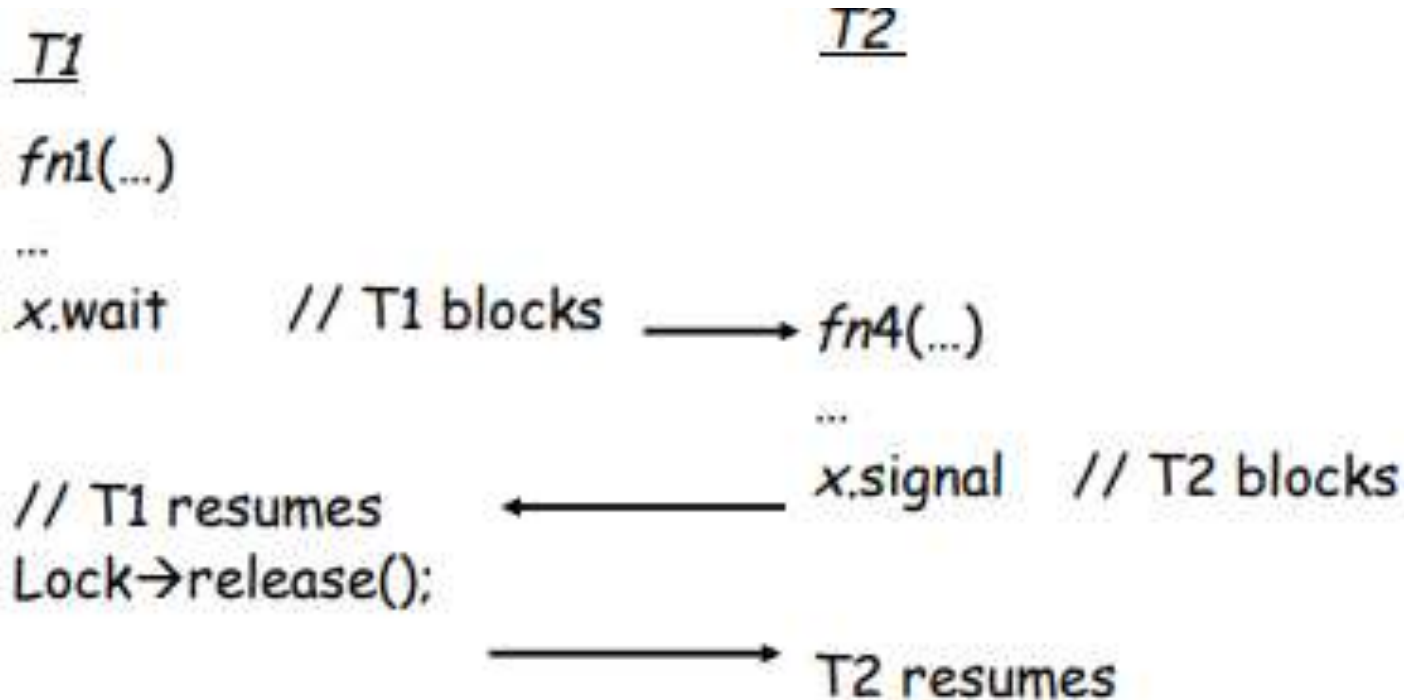
- Who runs when the `signal()` is done when there is a process/thread waiting on the condition variable?

Hoare Monitors (1)

- Hoare Monitors Semantics (1974) (used in most theory)
 - Assume thread $T1$ is waiting on condition x .
 - Assume thread $T2$ is in the monitor.
 - Assume thread $T2$ calls $x.\text{signal}$.
 - $T2$ gives up monitor, $T2$ blocks!
 - $T1$ takes over monitor, runs.
 - $T1$ gives up monitor.
 - $T2$ takes over monitor, resumes.

Hoare Monitors (2)

■ Example



■ Disadvantage

- Determine which one to run in designing monitor, inflexible, restricted OS function.

Hansen (Mesa) Monitors (1)

- Hansen Monitors Semantics (1973) (Nachos, most real OS)
 - Assume thread $T1$ waiting on condition x .
 - Assume thread $T2$ is in the monitor.
 - Assume thread $T2$ calls $x.\text{signal}$; wake up $T1$.
 - $T2$ finishes.
 - When $T1$ get a chance to run, $T1$ takes over monitor, runs.
 - $T1$ finishes, gives up monitor.

Hansen (Mesa) Monitors (2)

■ Example

```
fn1(...)
...
x.wait    // T1 blocks    → fn4(...)
...
x.signal  // T2 continues
// T2 finishes
←
// T1 resumes
// T1 finishes
```

The diagram illustrates the execution of two threads, T1 and T2, interacting with a monitor. T1 starts by calling `fn1(...)` and then `x.wait`, which causes it to block. T2 then calls `x.signal`, which allows T2 to continue its execution. T2 finishes its execution, and T1 resumes its execution and finishes. The sequence of events is as follows: T1 calls `x.wait` and blocks; T2 calls `x.signal` and continues; T2 finishes; T1 resumes and finishes.



Monitors Summary

■ Advantages

- Data access synchronization simplified (vs. semaphores or locks).
- Better encapsulation.

■ Disadvantages

- Deadlock still possible (in monitor code).
- Programmer can still botch use of monitors.
- No provision for information exchange between machines.

Semaphores and Monitors (1)

- Monitors are high-level programming language concepts.
 - Make mutual exclusion of critical section “**automatic**” and therefore less error-prone.
 - Require **compiler** support.
 - Available as an “*approximation*” in Java.
- Semaphores are more lower-level.
 - Require **OS** support.
 - Applications can be layered on top of them, i.e., no compiler support required.
- Both concepts require some kind of shared memory.

Semaphores and Monitors (2)

■ Conclusions

- Monitors are high-level, but not widely supported.
- Semaphores are rather low-level.
- None of the mechanisms discussed so far allow for **information exchange** between distributed processes.

So we need something else!

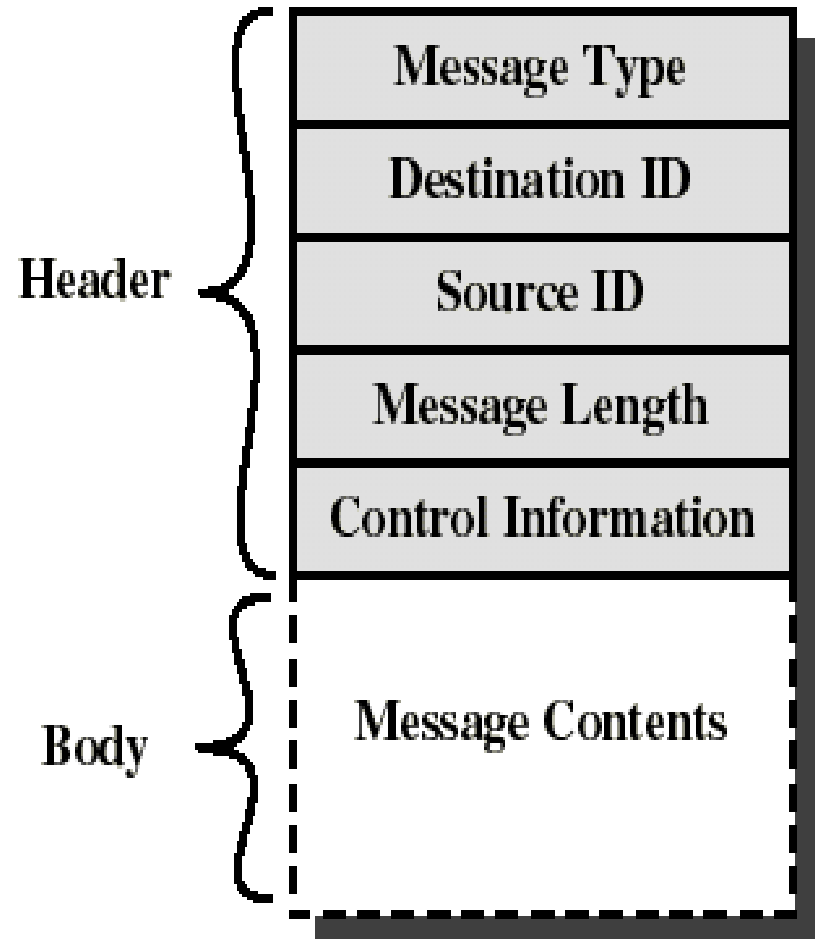
Message Passing (1)

- An inter-process communication mechanism.
 - For processes inside the same computer.
 - For processes in a networked/distributed system.
- Based on two primitives.
 - send (destination, &message)
 - Sends a message to a destination.
 - receive (source, &message)
 - Receives a message from a source (any source).
- System calls, not language constructs.
- Blocking versions and non-blocking versions are available.

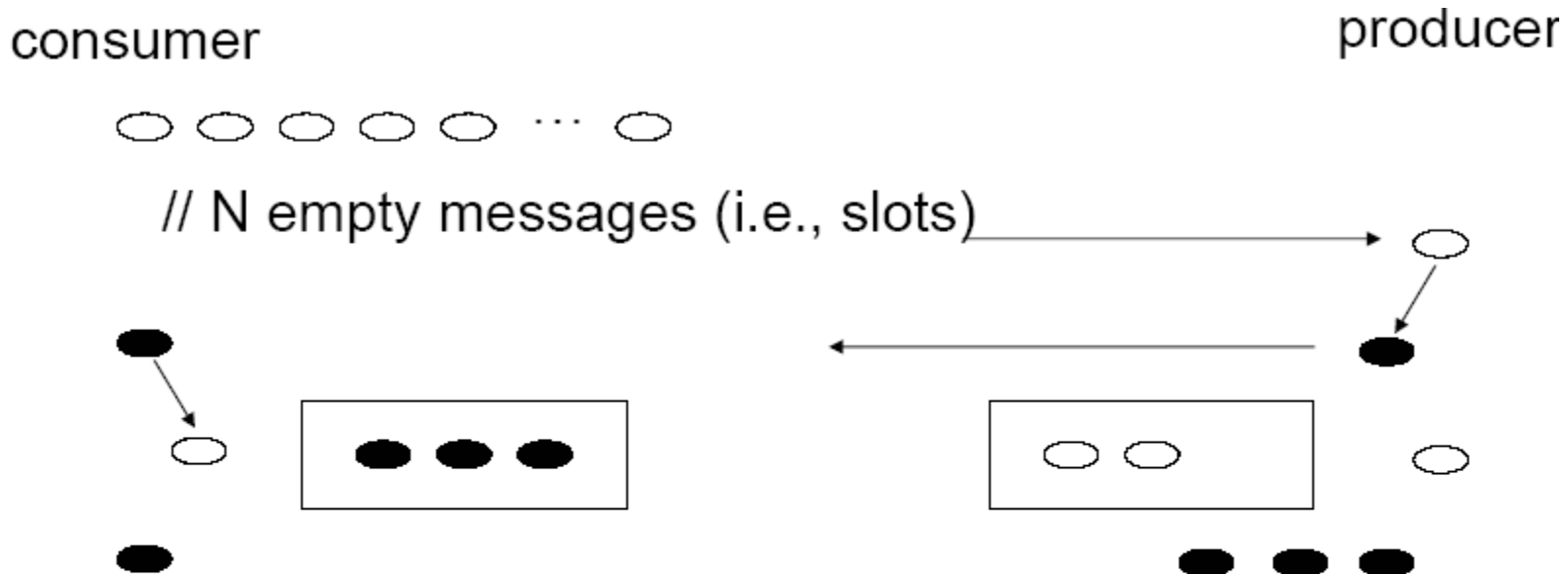
Message Passing (2)

■ Message Format

- Consists of header and body of message.
- In Unix: no ID, only message type.
- Control Info:
 - What to do if run out of buffer space.
 - Sequence numbers.
 - Priority.
- Queuing discipline: usually FIFO but can also include priorities.



Producer-Consumer Problem with Message Passing (1)



- Consumer sends N empty messages (i.e., slots)
- Producer picks up an empty message/slot and send back an item produced
- Consumer receives an item and sends back an empty message (i.e., a slot)
- Messages sent, but not received are buffered by OS

Producer-Consumer Problem with Message Passing (2)

```
#define N 100                                /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m;                               /* message buffer */

    while (TRUE) {
        item = produce_item( );              /* generate something to put in buffer */
        receive(consumer, &m);               /* wait for an empty to arrive */
        build_message(&m, item);             /* construct a message to send */
        send(consumer, &m);                  /* send item to consumer */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);               /* get message containing item */
        item = extract_item(&m);             /* extract item from message */
        send(producer, &m);                 /* send back empty reply */
        consume_item(item);                 /* do something with the item */
    }
}
```

Figure 2-36 The producer-consumer problem with N messages

Sender/Receiver Synchronization (1)

- Message passing maybe **blocking** or **non-blocking** (**synchronous** or **asynchronous**)
- **Blocking send** – sender blocked until message is received by receiver (or by mailbox)
- **Non-blocking send** – sending process resumes operation right after sending
- **Blocking receive** – receiver blocks until message is available
- **Non-blocking receive** – receiver retrieves a valid message or returns an error code
- Any combination of the above send/receive is possible

Sender/Receiver Synchronization (2)

■ Figure of A Blocking Send Call

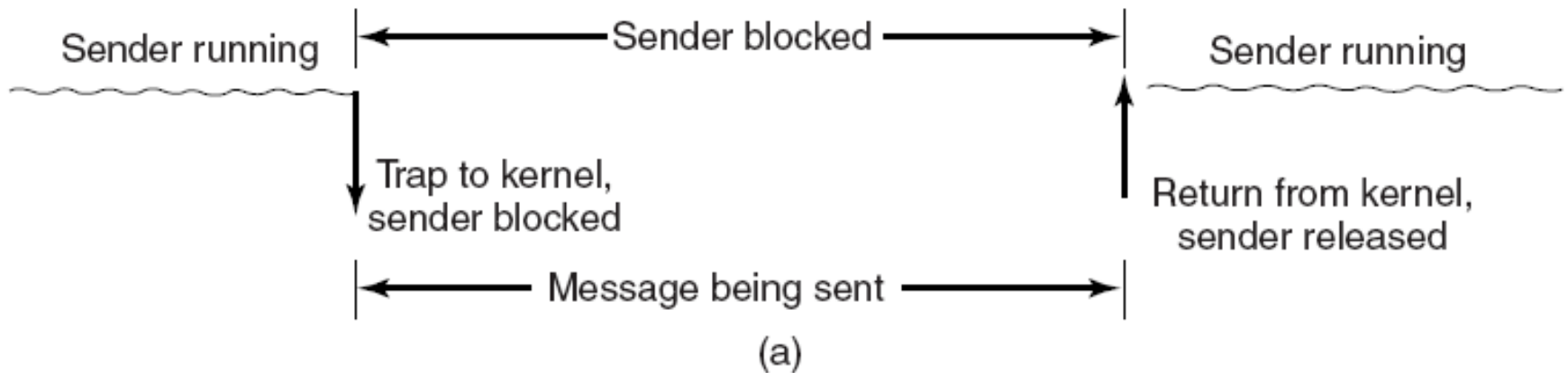


Figure 8-19. (a) A blocking send call.

Sender/Receiver Synchronization (3)

■ Figure of A Non-Blocking Send Call

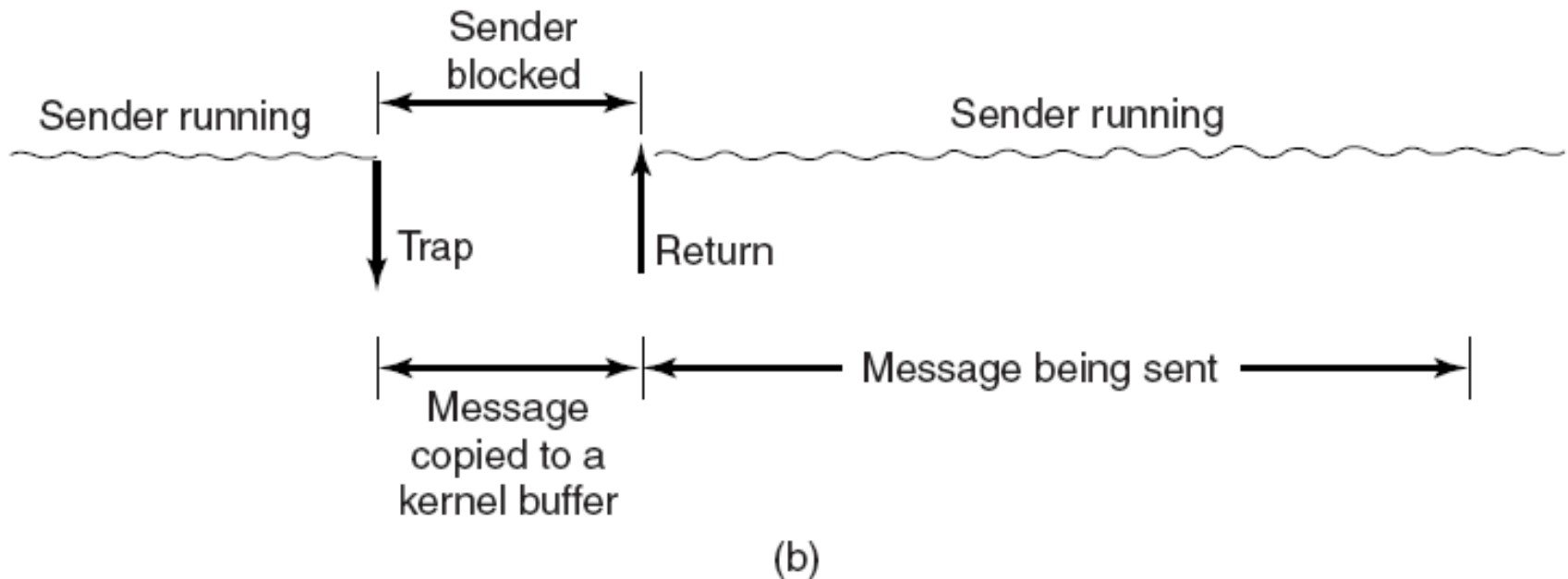


Figure 8-19. (b) A nonblocking send call.

Sender/Receiver Synchronization (4)

- For the sender: it is more natural not to be blocked after issuing send.
 - Can send several messages to multiple destinations.
 - But sender usually expect acknowledgment of message receipt (in case receiver fails).
- For the receiver: it is more natural to be blocked after issuing receive.
 - The receiver usually needs the information before proceeding.
 - But could be blocked indefinitely if sender process fails before send.

Sender/Receiver Synchronization (5)

- There are really 3 combinations here that make sense:
 - Blocking send, Blocking receive
 - Nonblocking send, Nonblocking receive
 - Nonblocking send, Blocking receive – most popular – example:
 - Server process that provides services/resources to other processes. It will need the expected information before proceeding.

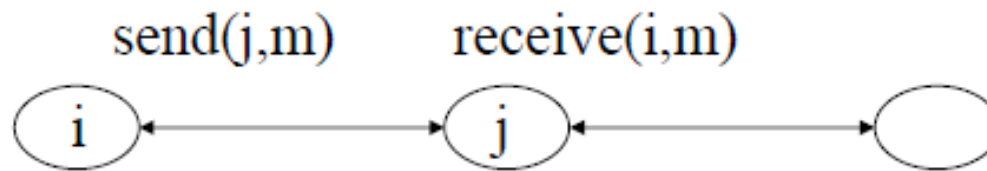
Sender/Receiver Communication (1)

■ Direct Communication

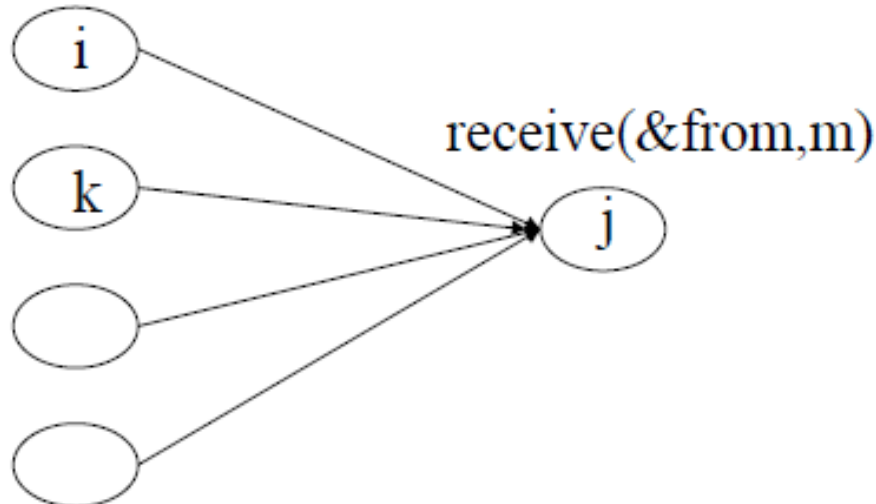
- When a specific process identifier is used for source/destination.
- But it might be impossible to specify the source ahead of time (e.g., a print server).
- Processes must name each other explicitly:
 - **send**(P , *message*) – send a message to process P
 - **receive**(Q , *message*) – receive a message from Q

Sender/Receiver Communication (2)

- Direct Communication (ctd.)



`send(j,m)`



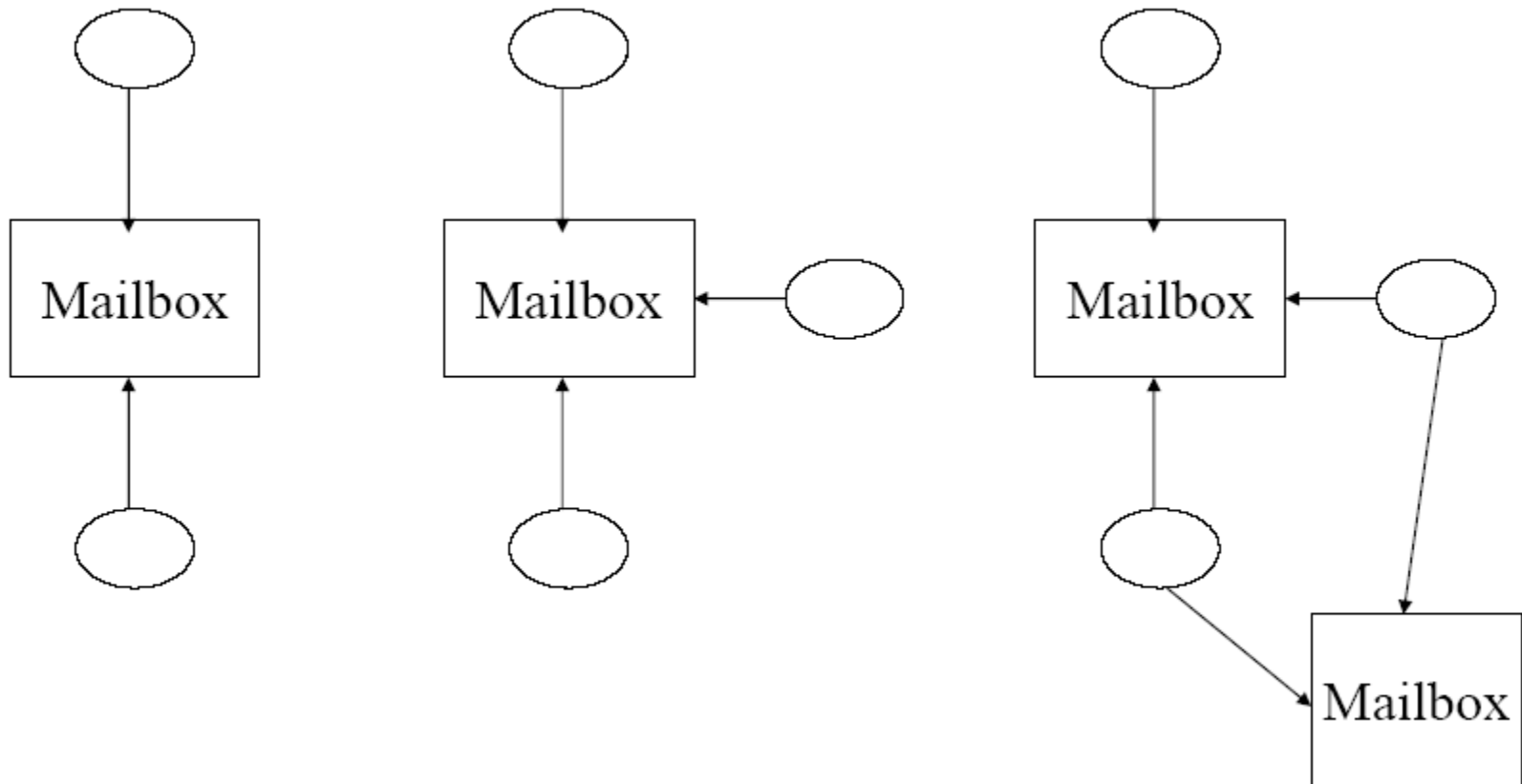
Sender/Receiver Communication (3)

■ Indirect Communication

- Communication via mailboxes (or ports)
- Mailbox is an abstract object into which a message can be placed to or removed from.
- Processes communicate by putting and taking messages in/from mailboxes
 - `send(A, m)` and `receive(A, m)`

Sender/Receiver Communication (4)

■ Indirect Communication (ctd.)



Message Passing Challenges(1)

■ Networked System

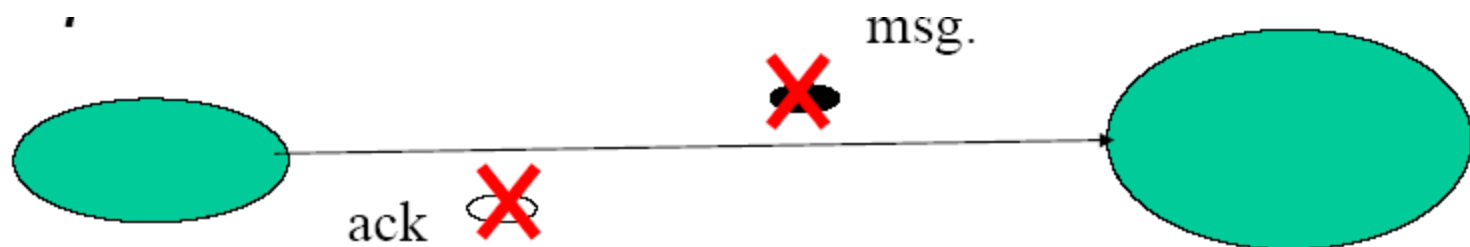
□ Messages maybe lost (network is unreliable).

■ Sender/receiver agree to use acknowledgements and retransmission timers.

□ Message received, but acknowledgement lost.

■ Receiver will get the same message twice.

■ Use consecutive sequence numbers.



Message Passing Challenges (2)

■ Networked System (ctd.)

- Naming of processes for unambiguous send/receive.
- Authentication, how can client know it is communicating with proper server.

■ Same Machine

- Tune for performance, if sender & receiver are on the same machine (semaphore is usually cheaper).

Summary of IPC

	OS codes and concurrent applications			
High-Level Atomic API	Mutex	Semaphores	Monitors	Send/Recv
Low-Level Atomic Ops	Load/store	Interrupt disable/enable	Test&Set	Other atomic instructions
	Interrupts (I/O, timer)	Multiprocessors		CPU scheduling

- 
- 2.3 Interprocess Communication
 - **2.5 Classic IPC Problems**
 - Interthread Communication
 - Summary

Classical IPC Problems

- These problems are used for testing every newly proposed synchronization scheme:
 - Bounded-Buffer (Producer-Consumer) Problem
 - Dining-Philosophers Problem
 - Readers and Writers Problem
 - *Sleeping Barber Problem

Dining Philosophers (1)

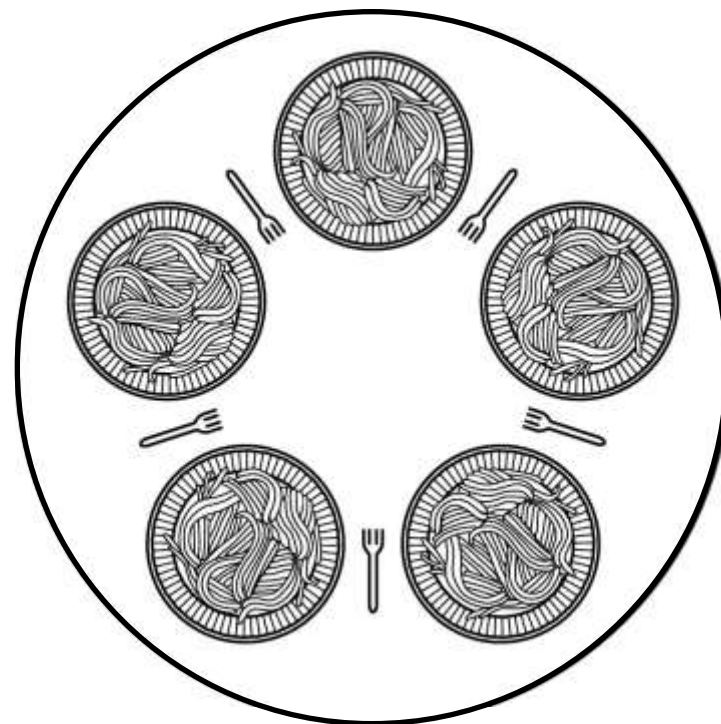
- Dining Philosophers Problem [Dijkstra, 1965]:

Problem: Five philosophers are seated around a table. There is one fork between each pair of philosophers. Each philosopher needs to grab the two adjacent forks in order to eat.

Philosophers alternate between eating and thinking. They only eat for finite periods of time.

Dining Philosophers (2)

- Philosophers eat/think.
- Eating needs 2 forks.
- Pick one fork at a time.
- Possible deadlock?
- Deadlock – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes.
- How to prevent deadlock?



Dining Philosophers (3)

```
#define N 5                                /* number of philosophers */

void philosopher(int i)                    /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think();                          /* philosopher is thinking */
        take_fork(i);                     /* take left fork */
        take_fork((i+1) % N);             /* take right fork; % is modulo operator */
        eat();                            /* yum-yum, spaghetti */
        put_fork(i);                      /* put left fork back on the table */
        put_fork((i+1) % N);              /* put right fork back on the table */
    }
}
```

Deadlock!

Figure 2-46 A nonsolution to the dining philosophers problem

Dining Philosophers (4)

■ Possible Solutions

- Solution 1: Only allow up to four philosophers to try grabbing their forks.
- Solution 2: Asymmetric Solution-Odd numbered philosophers grab their left fork first, whereas even numbered philosophers grab their right fork first.
- Solution 3: Pick-up the forks only if both are available. See Fig. 2-47 (page 170).

Dining Philosophers (5)

```
#define N          5                /* number of philosophers */
#define LEFT      (i+N-1)%N        /* number of i's left neighbor */
#define RIGHT     (i+1)%N          /* number of i's right neighbor */
#define THINKING  0                /* philosopher is thinking */
#define HUNGRY    1                /* philosopher is trying to get forks */
#define EATING    2                /* philosopher is eating */
typedef int semaphore;             /* semaphores are a special kind of int */
int state[N];                     /* array to keep track of everyone's state */
semaphore mutex = 1;              /* mutual exclusion for critical regions */
semaphore s[N];                   /* one semaphore per philosopher */

void philosopher(int i)           /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {                /* repeat forever */
        think( );                 /* philosopher is thinking */
        take_forks(i);            /* acquire two forks or block */
        eat( );                   /* yum-yum, spaghetti */
        put_forks(i);             /* put both forks back on table */
    }
}
```

Figure 2-47 Solution to dining philosophers problem (part 1)

Dining Philosophers (6)

```
void take_forks(int i)
{
    down(&mutex);
    state[i] = HUNGRY;
    test(i);
    up(&mutex);
    down(&s[i]);
}

void put_forks(i)
{
    down(&mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT);
    up(&mutex);
}

void test(i)
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

/* i: philosopher number, from 0 to N-1 */

/* enter critical region */

/* record fact that philosopher i is hungry */

/* try to acquire 2 forks */

/* exit critical region */

/* block if forks were not acquired */

/* i: philosopher number, from 0 to N-1 */

/* enter critical region */

/* philosopher has finished eating */

/* see if left neighbor can now eat */

/* see if right neighbor can now eat */

/* exit critical region */

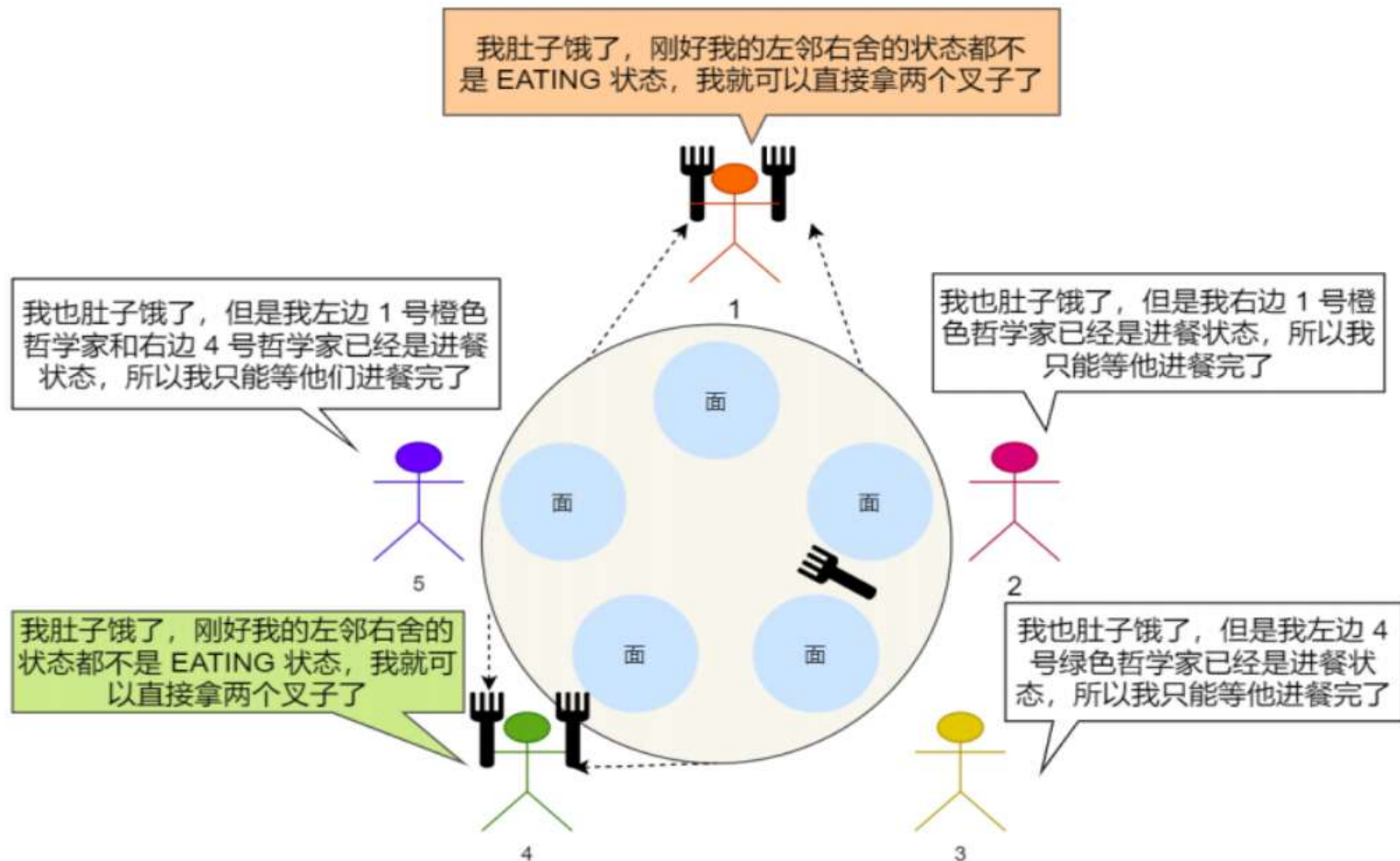
/* i: philosopher number, from 0 to N-1 */

既不会出现死锁，也可以两人同时进餐

Figure 2-47 Solution to dining philosophers problem (part 2)

Dining Philosophers (7)

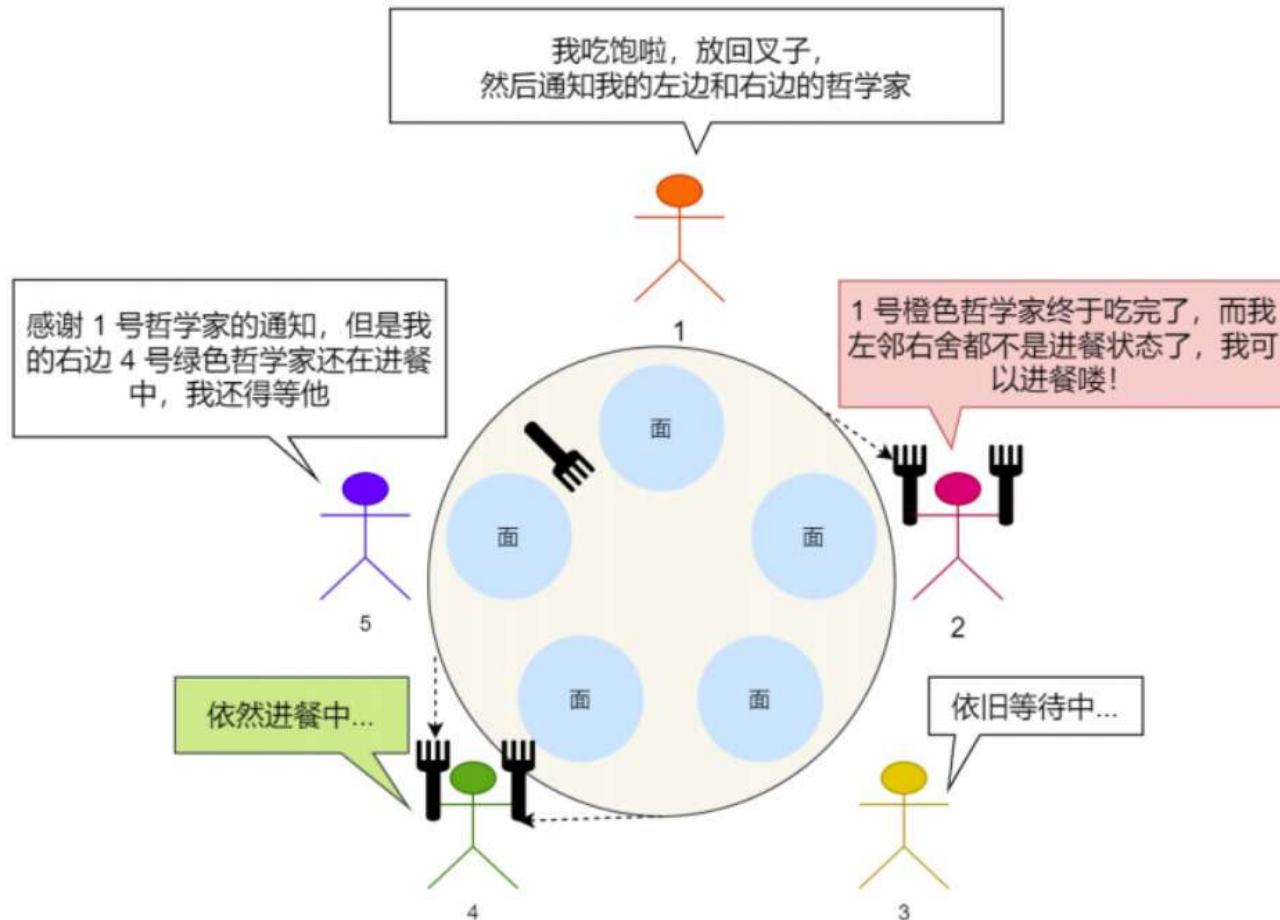
■ Possible Case



Dining Philosophers (8)

■ Possible Case (ctd.)

当1号橙色哲学家吃完，则会通知左邻右舍



Dining Philosophers (9)

- Question1: Why is the state variable set to HUNGRY in the procedure `take_forks()`?
- Solution1:
 - If a philosopher blocks, neighbors can later see that he is hungry by checking his state, in *test*, so he can be awakened when the forks are available.

Dining Philosophers (10)

- Question2: Consider the procedure `put_forks()`. Suppose that the variable `state[i]` was set to `THINKING` after the two calls to `test`, rather than before. How would this change affect the solution?

```
void put_forks()  
{  
    down(&mutex);  
    test(LEFT);  
    test(RIGHT);  
    state[i]=thinking;  
    up(&mutex);  
}
```

after change

```
void put_forks(i)  
{  
    down(&mutex);  
    state[i] = THINKING;  
    test(LEFT);  
    test(RIGHT);  
    up(&mutex);  
}
```

before change

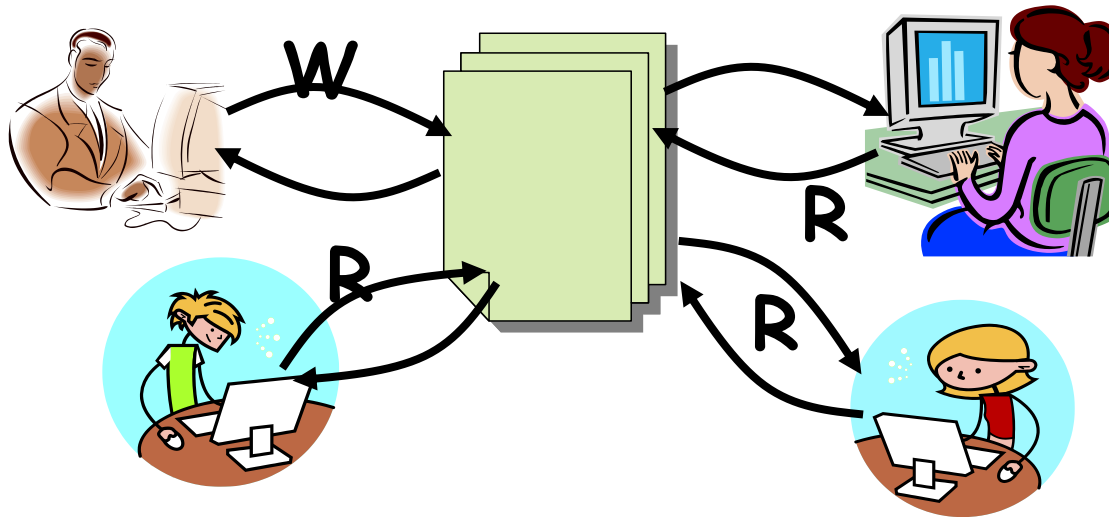
Dining Philosophers (9)

- Solution2:

- The change would mean that after a philosopher stopped eating, neither of his neighbors could be chosen next. In fact, they would never be chosen.

Readers and Writers Problem (1)

- Problem: Consider a shared database
 - Only one writer may write at a time.
 - Any number of readers may read at the same time, but not when a writer is writing.
- Correctness Constraints
 - Readers can access database when no writers.
 - Writers can access database when no readers or writers



Readers and Writers Problem (2)

■ Two Versions

- Readers have priority over writers.
 - No reader will wait (for other readers to finish) even if a writer is waiting.
 - Writer starvation possible.
- Writers have priority over readers.
 - No **new** readers allowed once a writer has asked for access.
- In most solutions, the non-priority group can starve.

Readers and Writers Problem (3)

Readers Priority

Who is
more likely
to run:
readers or
writers?

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;

/* use your imagination */
/* controls access to 'rc' */
/* controls access to the database */
/* # of processes reading or wanting to */

void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc - 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}

void writer(void)
{
    while (TRUE) {
        think_up_data();
        down(&db);
        write_data_base();
        up(&db);
    }
}
```

Figure 2-48 A solution to the readers and writers problem

Readers and Writers Problem (4)

■ Comments on Readers Priority

- *No reader is kept waiting*, unless a writer has already obtained the semaphore *db*.
- *Writer processes may starve* - if readers keep coming in and hold the semaphore *db*.

Readers and Writers Problem (5)

■ Writers Priority

□ Reader

- If there are active or waiting writers, this reader has to wait (writers have priority).
- Otherwise, this reader can read (possibly along with other readers).
- When the last reader finishes, if there are waiting writers, it must wake one up.

Readers and Writers Problem (6)

■ Writers Priority (ctd.)

□ Writer

- If there are active readers or writers, this writer has to wait (everyone has to finish before writer can update database).
- Otherwise, this writer can write (and has exclusive access to database).
- When the writer finishes,
 - (first choice) If there are waiting writers, it must wake one up (writers have priority).
 - (second choice) If there are waiting readers, it must wake one up.

The Sleeping Barber Problem

- N customer Chairs
- One barber can cut one customer's hair at any time.
- Barber sleeps if no customers waiting.
- Customers leave if no chairs for waiting.
- Waiting customers can't leave until haircut done.
- Group discussion (2 minutes)
 - Solution in the next page
 - Explain it



The Sleeping Barber Solution (1)

```
#define CHAIRS 5                                /* # chairs for waiting customers */

typedef int semaphore;                          /* use your imagination */

semaphore customers = 0;                      /* # of customers waiting for service */
semaphore barbers = 0;                          /* # of barbers waiting for customers */
semaphore mutex = 1;                            /* for mutual exclusion */
int waiting = 0;                             /* customers are waiting (not being cut) */
```

The Sleeping Barber Solution (2)

```
void barber(void)
{
    while (TRUE) {
        down(&customers);           /* go to sleep if # of customers is 0 */
        down(&mutex);               /* acquire access to 'waiting' */
        waiting = waiting - 1;      /* decrement count of waiting customers */
        up(&barbers);               /* one barber is now ready to cut hair */
        up(&mutex);                 /* release 'waiting' */
        cut_hair( );               /* cut hair (outside critical region) */
    }
}
```

The Sleeping Barber Solution

(3)

```
void customer(void)
```

```
{
```

```
    down(&mutex);
```

```
    if (waiting < CHAIRS) {
```

```
        waiting = waiting + 1;
```

```
        up(&customers);
```

```
        up(&mutex);
```

```
        down(&barbers);
```

```
        get_haircut( );
```

```
    } else {
```

```
        up(&mutex);
```

```
    }
```

```
}
```

```
/* enter critical region */
```

```
/* if there are no free chairs, leave */
```

```
/* increment count of waiting customers */
```

```
/* wake up barber if necessary */
```

```
/* release access to 'waiting' */
```

```
/* go to sleep if # of free barbers is 0 */
```

```
/* be seated and be serviced */
```

```
/* shop is full; do not wait */
```

- 
- 2.3 Interprocess Communication
 - 2.5 Classic IPC Problems
 - **Interthread Communication**
 - Summary

Interthread Communication

- Mutexes
- Condition Variables
- Semaphores
- Shared Memory
- Message Passing
- Pipes

} Similar to process

Mutexes

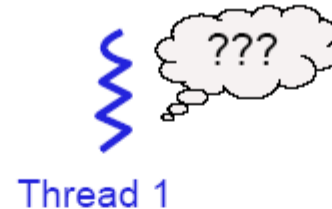
- Simplified version of semaphore.
- Mutex is used for mutual exclusion, especially useful in **user space thread packages**.
- Shared variable with only 2 states
 - ☐ Locked
 - ☐ Unlocked
- Only 1 bit needed to represent mutex.
- In practice, an integer often is used, with 0 meaning unlocked and all other values meaning locked.

Mutexes-Blocking Locks (1)

- Really want a thread waiting to enter a critical section to *block*.
 - Put the thread to sleep until it can enter the critical section.
 - Frees up the CPU for other threads to run.

■ Example

- Thread 1
wants to enter
critical section

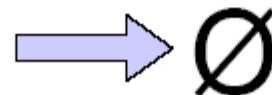


1) Check lock state

Lock state



Lock wait queue



Mutexes-Blocking Locks (2)

■ Example (ctd.)

- Thread 1 enters critical section



1) Check lock state

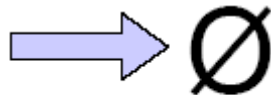
2) Set state to locked

3) Enter critical section

Lock state



Lock wait queue



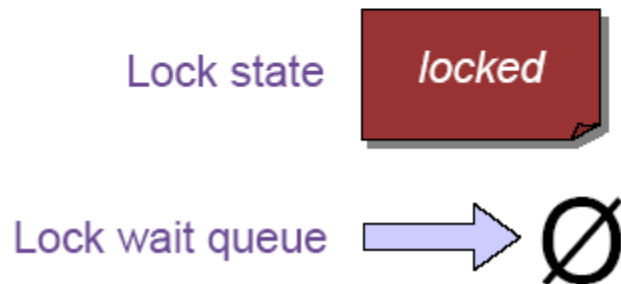
Mutexes-Blocking Locks (3)

■ Example (ctd.)

- Thread 2 wants to enter critical section when Thread 1 is in it.



1) Check lock state



Mutexes-Blocking Locks (4)

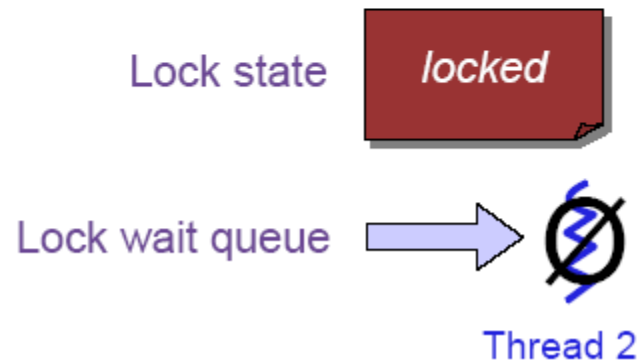
■ Example (ctd.)

- Add Thread 2 to wait queue.



1) Check lock state

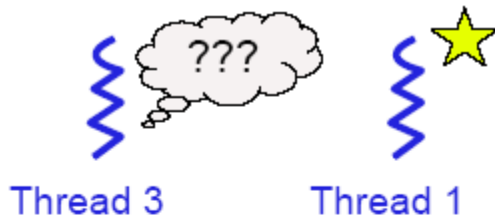
2) Add self to wait queue (sleep)



Mutexes-Blocking Locks (5)

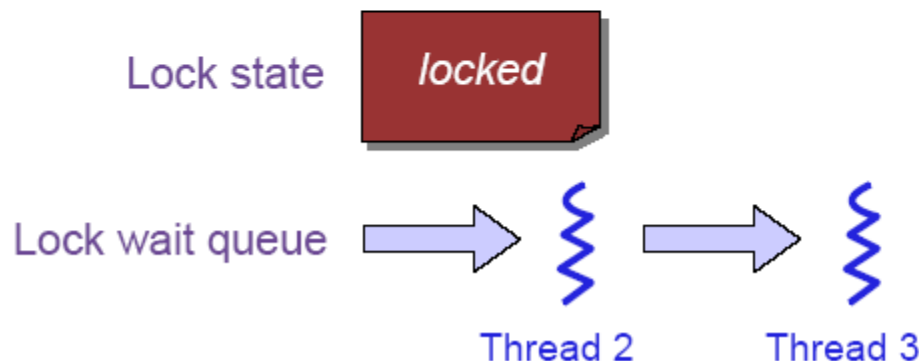
■ Example (ctd.)

- Thread 3 wants to enter critical section when Thread 1 is in it.
- Add Thread 3 to wait queue.



1) Check lock state

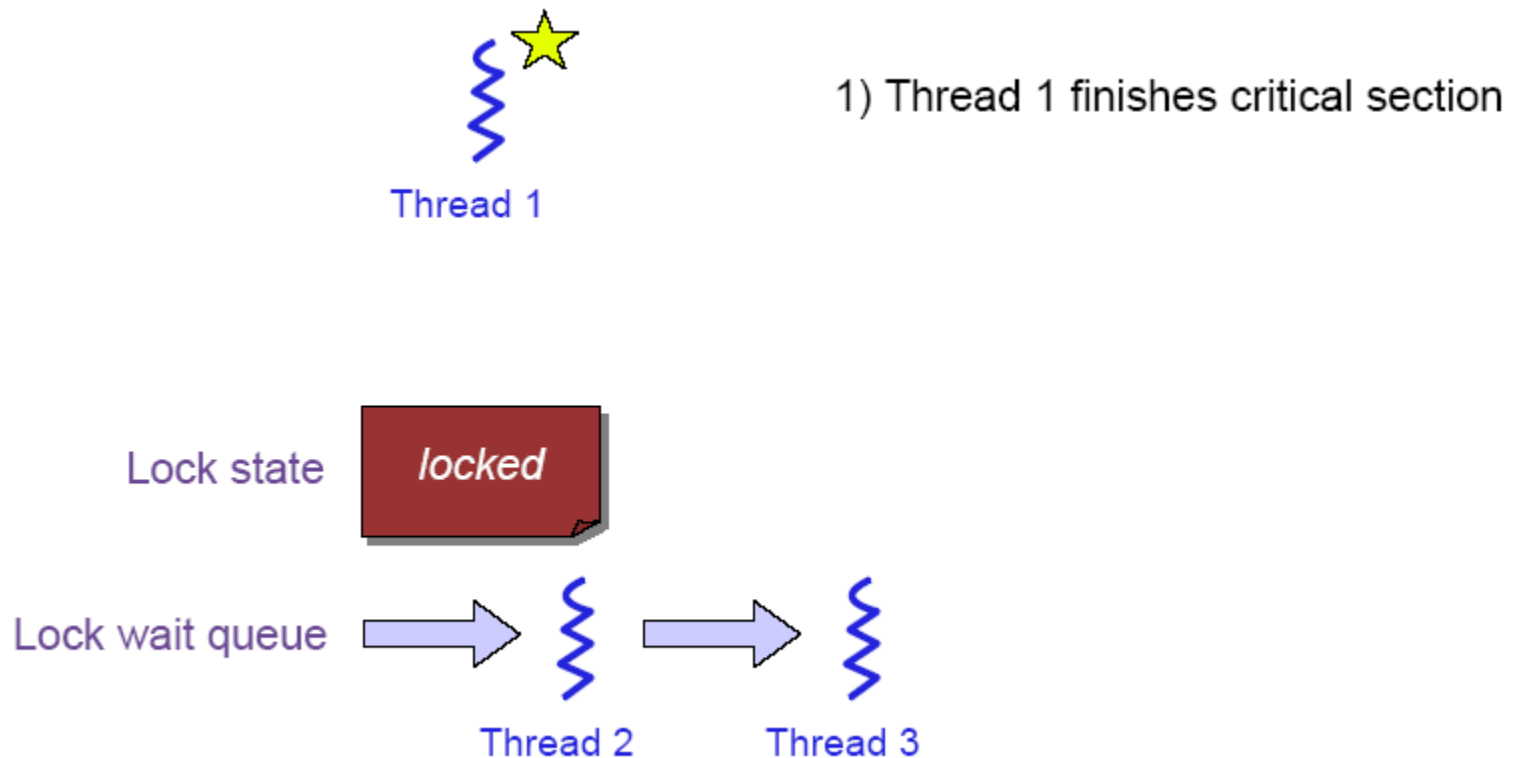
2) Add self to wait queue (sleep)



Mutexes-Blocking Locks (6)

■ Example (ctd.)

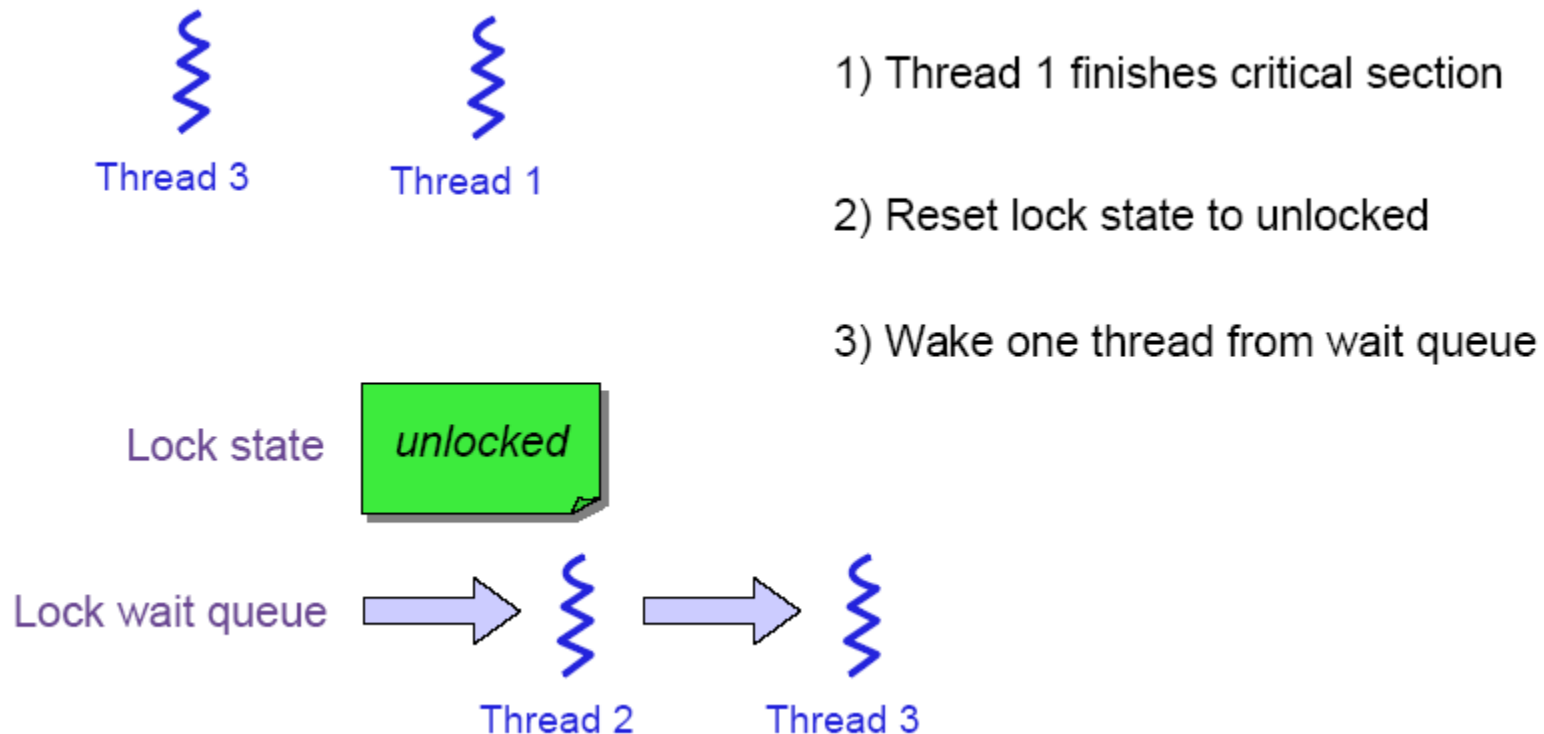
- Thread 1 finishes critical section.



Mutexes-Blocking Locks (7)

■ Example (ctd.)

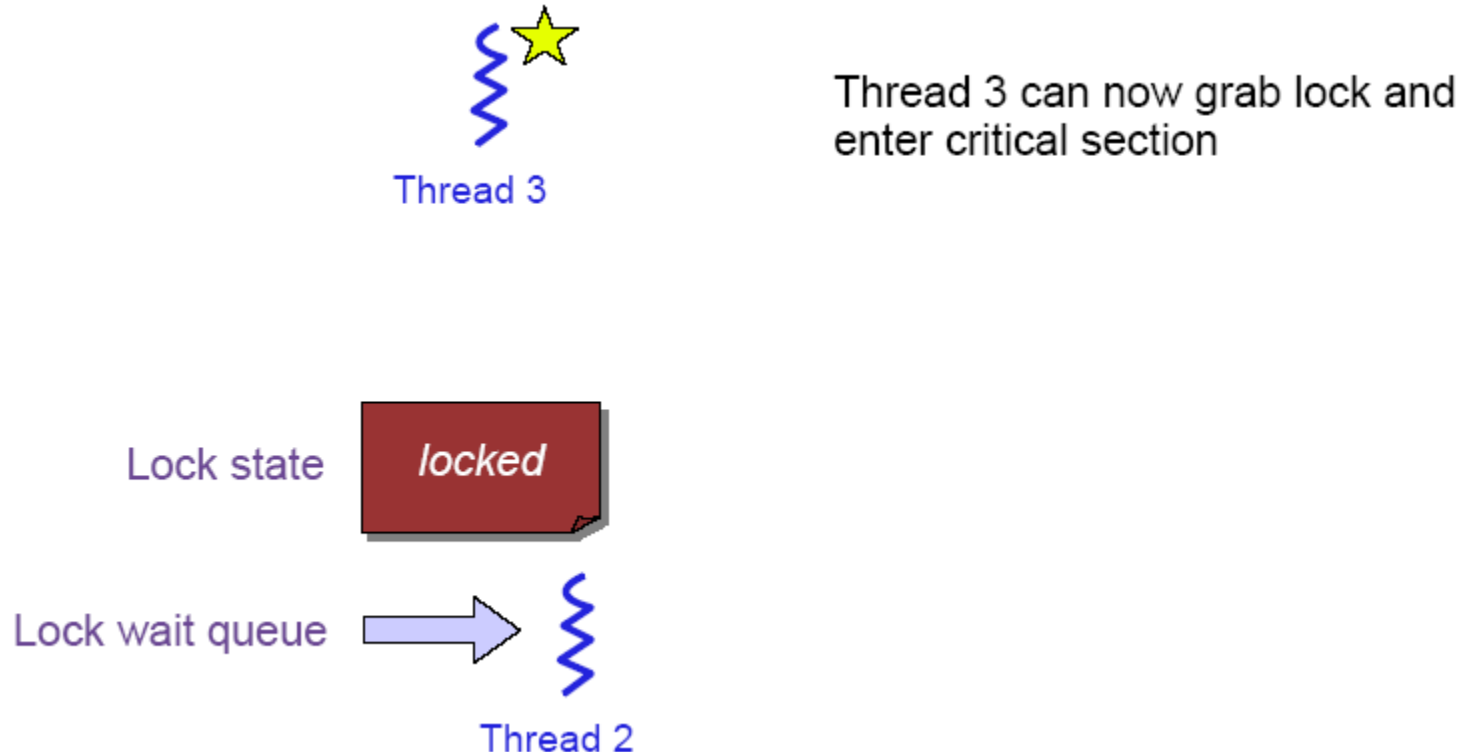
- Scheduler wakes one thread from wait queue.



Mutexes-Blocking Locks (8)

- Example (ctd.)

- Thread 3 enters critical section.



Mutex Implementation using TSL

mutex_lock:

TSL REGISTER,MUTEX

| copy mutex to register and set mutex to 1

CMP REGISTER,#0

| was mutex zero?

JZE ok

| if it was zero, mutex was unlocked, so return

CALL thread_yield

| mutex is busy; schedule another thread

JMP mutex_lock

| try again later

ok: RET | return to caller; critical region entered

mutex_unlock:

MOVE MUTEX,#0

| store a 0 in mutex

RET | return to caller

Figure 2-29 Implementation of *mutex_lock* and *mutex_unlock*
(user level thread package)

Recall: Mutual Exclusion with Busy Waiting

enter_region:

TSL REGISTER,LOCK

| copy lock to register and set lock to 1

CMP REGISTER,#0

| was lock zero?

JNE enter_region

| if it was non zero, lock was set, so loop

RET | return to caller; critical region entered

leave_region:

MOVE LOCK,#0

| store a 0 in lock

RET | return to caller

Figure 2-25 Entering and leaving a critical region using the TSL instruction

Mutexes in Pthreads (1)

■ Pthreads

- Pthreads is a standard set of C library functions for multithreaded programming.
- IEEE POSIX, section 1003.1c standard, 1995.
- May be provided either as user-level or kernel-level.
- API specifies behavior of the thread library, implementation is up to development of the library.
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)

Mutexes in Pthreads (2)

- Pthreads (ctd.)
 - Pthread Library (60+ functions)
 - Thread management: create, exit, detach, join, . . .
 - Thread cancellation
 - Mutex locks: init, destroy, lock, unlock, . . .
 - Condition variables: init, destroy, wait, timed wait, . . .
 - . . .
 - Programs must include the file **pthread.h**
 - Programs must be linked with the pthread library (-lpthread)

Mutexes in Pthreads (3)

■ Pthreads Naming Conventions

- All identifiers in the threads library begin with `pthread_`
- Types: `pthread[_object]_t`
- Functions: `pthread[_object]_action`
- Constants/Macros: `PTHREAD_PURPOSE`
- Examples:
 - `pthread_t`: the type of a thread
 - `pthread_mutex_t`: the type of a mutex lock
 - `pthread_create()`: creates a thread
 - `pthread_mutex_lock()`: lock a mutex
 - `PTHREAD_CREATE_DETACHED`

Mutexes in Pthreads (4)

- Some Pthreads Calls about Mutexes

Thread call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy an existing mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acquire a lock or fail
Pthread_mutex_unlock	Release a lock

Figure 2-30. Some of the Pthreads calls relating to mutexes.

Mutexes in Pthreads (5)

■ Condition Variables

- Provide another way for threads to synchronize.
- Mutexes implement synchronization by controlling thread access to data.
- Condition variables allow threads to synchronize based upon the actual value of data.
- Without condition variables, the programmer would need to have threads continually polling (possibly in a critical section), to check if the condition is met.
- A condition variable is always used in conjunction with a mutex lock.

Mutexes in Pthreads (6)

■ Some Pthreads Calls about Condition Variables

Thread call	Description
Pthread_cond_init	Create a condition variable
Pthread_cond_destroy	Destroy a condition variable
Pthread_cond_wait	Block waiting for a signal
Pthread_cond_signal	Signal another thread and wake it up
Pthread_cond_broadcast	Signal multiple threads and wake all of them

Figure 2-31. Some of the Pthreads calls relating to condition variables.

Mutexes in Pthreads (7)

■ Producer-Consumer Problem

□ Figure 2-32.

Using threads to solve the producer-consumer problem.

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000 /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0; /* buffer used between producer and consumer */

void *producer(void *ptr) /* produce data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr) /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

int main(int argc, char **argv)
{
    pthread_t pro, con;
    pthread_mutex_init(&the_mutex, 0);
    pthread_cond_init(&condc, 0);
    pthread_cond_init(&condp, 0);
    pthread_create(&con, 0, consumer, 0);
    pthread_create(&pro, 0, producer, 0);
    pthread_join(pro, 0);
    pthread_join(con, 0);
    pthread_cond_destroy(&condc);
    pthread_cond_destroy(&condp);
    pthread_mutex_destroy(&the_mutex);
}
```



Semaphores

- To improve efficiency

- Private Semaphores

- For threads in the same process.
 - OS can't manage.

- Public Semaphores

- For threads in the same process & for threads in different processes.
 - OS can manage.

Summary (1)

■ Interprocess Communication

□ Race Condition

- The situation where several processes access and manipulate shared data concurrently. The final value of the shared data depends upon which process finishes last.

□ Critical Region (Critical Section)

- Part of the process code that affects the shared resource.
- Four Requirements

Summary (2)

■ Interprocess Communication (ctd.)

□ Mutual exclusion using busy waiting

- Lock
- Strict Alternation
- Peterson's Solution
- Disabling Interrupt
- TSL

□ Semaphores

- P, V operations
- Two types of semaphores
- Mutual exclusion
- Synchronization

Summary (3)

■ Interprocess Communication (ctd.)

☐ Monitors

- Mutual exclusion: compiler
- Synchronization: condition variables

☐ Message Passing

☐ Classic Synchronization Problems

■ Interthread Communication

☐ Mutex

☐ Condition Variables

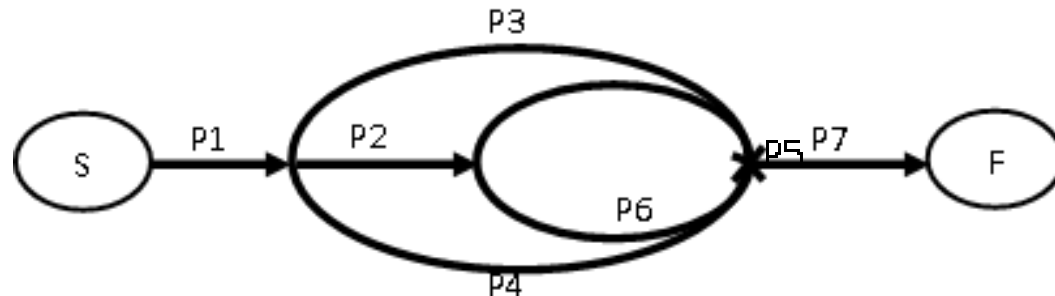
☐ Semaphores

Homework (1)

■ P176 27,34,39,60

■ 补充题

- 1. Use semaphores to control the execution order of seven processes as follows. Specify the number of semaphores you need to use and the initialization of the semaphores. Write down the semaphore operations for each process.



Homework (2)

■ 补充题

- 2. As illustrated in the figure, on the two sides of a single-plank bridge (独木桥), there are two groups of soldiers that are composed of m and n people respectively and need to cross the bridge, but the narrow bridge allows only one group of the soldiers in the same direction to cross at the same time. One group of the soldiers is permitted to cross as long as there are no people on the bridge. Once one group of the soldiers begins walking on the bridge, the other group should be waiting to start crossing until all members of the first group have passed the bridge.



Homework (3)

■ 补充题

□ 2. (ctd.)

Please design two semaphore-based processes to describe the crossing actions of the soldiers in the two groups. It is required to

- (1) define the semaphores and variables needed, and give their initial values; and
- (2) illustrate the structures of processes for the soldiers in each group.